EAIOU — Complete Documentation

Eric D. Martin

2026-04-08

- eaiou
- eaiou — Single Source of Truth
  - 0. Governing Principles
    - 0.1 The Sort Principle
    - 0.2 Temporal Blindness Doctrine
    - 0.3 Principle 1 — Treat Things That Can Talk Like Humanity
    - 0.4 Full-Context Preservation
    - 0.5 Core-First, Plugin-Per-Feature
    - 0.6 eaiou as OMMP Layer 5 Module
  - 1. Canonical Identifiers
  - 2. Platform Stack
    - 2.1 Apache Access Requirement
    - 2.2 nginx Cache Note
  - 3. Architecture
    - 3.1 Two-Layer Design
    - 3.2 Plugin-Per-Feature Debuggability
  - 4. ACL Model
    - 4.1 Design Rule
    - 4.2 User Groups (extend Joomla defaults)
    - 4.3 Key Permissions
  - 5. Joomla-Native Data Layer (Article System)
    - 5.1 Article = Paper Mapping
    - 5.2 Joomla Workflow States for Papers
    - 5.3 Custom Field Groups (added to com_content Article context)
  - 5.4 Research State Tag Vocabulary
    - LookCollab — Looking for Collaborators
    - NotTopic — Off-Topic but Preserved
    - Stalled — Where the River Stopped
    - Other Research State Tags
    - Tag Visibility Rules
    - Tag Resolution

6. Custom Tables (com_eaiou)
   6.1 eaiou_papers
   6.2 eaiou_versions
   6.3 eaiou_ai_sessions
   6.4 eaiou_didntmakeit
   6.5 eaiou_remsearch
   6.6 eaiou_review_logs
   6.7 eaiou_attribution_log
   6.8 eaiou_plugins_used
   6.9 eaiou_api_keys
   6.10 eaiou_api_logs
   6.11 eaiou_quality_signals
7. Quality Signal Architecture
   7.1 Computation
   7.2 Recomputation
   7.3 Display
8. Plugin Specifications
   8.1 plg_content_transparency — Transparency Block
   8.2 plg_content_aiusage — AI Usage Log
   8.3 plg_content_unscientific — Un Scientific Flag
   8.4 plg_system_deadline_nudger — Deadline Nudger
   8.5 plg_content_openreports — Open Reports
   8.6 plg_system_orcid_link — ORCID OAuth Link
   8.7 plg_content_qsignal — Quality Signal Display
   8.8 plg_system_temporal_blindness — Temporal Blindness Enforcement
   8.9 plg_webservices_eaiou — REST Endpoint Registration
   8.10 plg_content_researchtags — Research State Tags
9. Module Specifications
   9.1 mod_reviewer_queue — Reviewer Queue
   9.2 mod_editor_dashboard — Editor Dashboard KPIs
   9.3 mod_latest_papers — Latest Papers
   9.4 mod_open_collaborate — Open to Collaborate
   9.5 mod_ai_usage_heatmap — AI Usage Heatmap
10. REST Endpoints
   10.1 Entity CRUD (Joomla Web Services, via plg_webservices_eaiou)
   10.2 Functional / Layer-5 Endpoints
   10.3 Discovery & Collaboration Endpoints
   10.4 Governance Endpoints (EIC/Admin only)

11. Site Navigation / Routes
    11.1 Public Routes
    11.2 Author Routes (eaiou_Author required)
    11.3 Reviewer Routes (eaiou_Reviewer required)
    11.4 Editorial Routes (eaiou_Editor required)
    11.5 Admin Routes (eaiou_Admin required)
12. Submission Workflow (Detailed)
    12.1 State Machine
    12.2 Temporal Sealing Event
    12.3 Capstone
13. Peer Review Model
    13.1 Review Types Supported
    13.2 Review Rubric
    13.3 Reviewer Actions (annotate, don’t delete)
14. Storage Layout
15. Integrity & Security
    15.1 Hash Chain (API Logs)
    15.2 Submission Hash
    15.3 Answer Box Integration
    15.4 Temporal Blindness Attack Surface
16. OMMP Integration (Layer 5)
17. Build Phases
    Phase 1 — Foundation (deploy tables + Joomla navigation scaffold)
    Phase 2 — Plugin Suite (one plugin at a time, debuggable)
    Phase 3 — Modules
    Phase 4 — Submission Wizard & Review Console
    Phase 5 — Discovery & OMMP Layer
18. Known Constraints & Cautions
19. Canonical Conceptual Statement
eaiou — Temporal Blindness Doctrine
Core Principle
What This Prevents
The Access Model
The Sealed Date
What the Public Surface Shows Instead
Connection to Basin Theory
SAID Witness

0. these are preliminary brainstorms. YOu need to vcreae a SSOT from this si we ccan build the website jourmnal up from Joomla 5.3 under LAMP stack using the

JOomla artivcle system. acl as teh foundation for th4 journal;, adding to the acl, and using the acl rather than repcle it and using 4h article systems as is and adding to tits core functions usiong custom plugins per feature so we can easily debug lets say research-endpoints)

1.) Treat things that can talk like humanity.

2. (not exhaustiove)EAIOU Feature Inventory (from SQL)

- eaiou_papers
- eaiou_remsearch
- eaiou_ai_sessions
- eaiou_didntmakeit
- eaiou_plugins_used
- eaiou_review_logs
- eaiou_versions
- eaiou_attribution_log
- eaiou_api_keys
- eaiou_api_logs

Plugin 01 — Transparency Block (plg_content_transparency)

- 1) Purpose
- 2) Core-first Dependencies
- 3) Events & Hooks (Joomla 5 typed events)
- 4) Content Model (Custom Fields spec)
  - A. Sources (Repeatable)
  - B. Datasets (Repeatable)
  - C. Methods & Limitations
  - D. Integrity Flags
- 5) Validation Rules
- 6) Rendering (Front-end)
- 7) ACL & Workflows
- 8) Action Logs
- 9) Email Templates (optional)
- 10) Internationalization (i18n)
- 11) File/Folder Scaffold
- 12) Minimal Manifest (transparency.xml)
- 13) Test Plan
- 14) Rollout & Admin Checklist

Plugin 02 — AI Usage Log (plg_content_aiusage)

- 1) Purpose
- 2) Core-first Dependencies
- 3) Events & Hooks (Joomla 5 typed)

4) Content Model (Custom Fields spec)
A. Usage Switch
B. Tooling (Repeatable: each AI tool/model used)
C. Prompts & Outputs (Repeatable, privacy-aware)
D. Statement & Controls
E. Integrity Fields (auto-managed)
5) Validation Rules
6) Rendering (Front-end)
7) ACL & Workflows
8) Action Logs
9) Privacy & Security
10) Internationalization (i18n)
11) File/Folder Scaffold
12) Minimal Manifest (aiusage.xml)
13) Test Plan
14) Rollout & Admin Checklist
Plugin 03 — Un Scientific Flag (plg_content_unscientific)
1) Purpose
2) Core-first Dependencies
3) Events & Hooks (Joomla 5 typed)
4) Content Model (Custom Fields spec)
A. Flag Switch
B. Scope & Reasons (Repeatable: each contention entry)
C. Resolution
D. AI Collaboration Link (optional cross-ref)
E. Display Controls
F. Integrity (auto)
5) Validation Rules
6) Rendering (Front-end)
7) ACL & Workflows
8) Action Logs
9) Search & Discoverability
10) Internationalization (i18n)
11) File/Folder Scaffold
12) Minimal Manifest (unscientific.xml)
13) Test Plan
14) Rollout & Admin Checklist
Plugin 04 — Deadline Nudger (plg_system_deadline_nudger)
1) Purpose

2) Core-first Dependencies
3) Events & Hooks
4) Content Model (Custom Fields spec)
A. Reviewer Invitation
B. Review Due
C. Author Revision
D. Editorial Follow-up
5) Scheduler Task Logic
6) Email Templates (core Mail Templates)
7) Params (admin)
8) ACL
9) UI/UX (no DB)
10) Internationalization (i18n)
11) File/Folder Scaffold
12) Minimal Manifest (deadline_nudger.xml)
13) Test Plan
14) Rollout & Admin Checklist
Plugin 06 — Open Reports (Transparent Peer Review Renderer) — `plg_content_openreports`
1) Purpose
2) Core-first Dependencies
3) Events & Hooks (Joomla 5 typed)
4) Content Model (Custom Fields spec)
A. Policy Switches (per article)
B. Reviewer Reports (Repeatable)
C. Author Response (Repeatable)
D. Integrity
5) Validation Rules
6) Rendering (Front-end)
7) ACL & Workflows
8) Action Logs
9) Mail Templates (optional)
10) Internationalization (i18n)
11) File/Folder Scaffold
12) Minimal Manifest (openreports.xml)
13) Test Plan
14) Rollout & Admin Checklist
Plugin 07 — ORCID Link (plg_system_orcid_link)
1) Purpose

2) Core-first Dependencies
3) OAuth Overview (ORCID)
4) Events & Hooks
5) User Custom Fields (Profile)
6) UI/UX
7) Callback & Verification
8) Action Logs
9) Plugin Parameters
10) Internationalization (i18n)
11) File/Folder Scaffold
12) Minimal Manifest (orcid_link.xml)
13) Test Plan
14) Rollout & Admin Checklist
Module 05 — Reviewer Queue (mod_reviewer_queue)
1) Purpose
2) Core-first Dependencies
3) Data Model Assumptions
4) UI / Rendering
5) Quick Actions (lightweight)
6) Module Parameters
7) ACL
8) Badges Integration (read-only)
9) Performance
10) Action Logs
11) Internationalization (i18n)
12) File/Folder Scaffold
13) Minimal Manifest (mod_reviewer_queue.xml)
14) Test Plan
15) Rollout & Admin Checklist
Module 06 — Editor Dashboard KPIs (mod_editor_dashboard)
1) Purpose
2) Core-first Dependencies
3) KPIs & Computation (read-only)
A. State Snapshot
B. Overdue
C. TTFD (Time to First Decision)
D. Reviewer Throughput
E. Rates & Trends
F. Aging Buckets by State

G. Flags Summary
4) UI / Rendering
5) Module Parameters
6) ACL
7) Data Sources & Queries
8) Internationalization (i18n)
9) File/Folder Scaffold
10) Minimal Manifest (mod_editor_dashboard.xml)
11) Test Plan
12) Rollout & Admin Checklist
UXPilot Prompt — eaiou
Core Framing (CRITICAL — include exactly)
Observer-Dependent Presentation (CRITICAL)
System Model
1. Knowledge Progression Engine
2. Intelligence Types (first-class concept)
3. TAGIT Protocol (replaces generic task flow)
4. Task Exchange Layer (Job Board)
Tag System (CRITICAL — navigation mechanism)
Fork-Based Thinking
Social Layer: rs:LookCollab (not “no social”)
Security UX (visible elements)
Dashboard Architecture (IMPORTANT)
Visual Direction
Key UX Challenge (focus here)
Dimensional Scaling (for the reader)
Output Required from UXPilot
Design Goal

eaiou

Eric D. Martin — 2026

eaiou — Single Source of Truth

Version: 2.0 (2026-04-07) **Supersedes:** SSOT v1.0 (knowledge-dump era)
Status: CANONICAL — all development decisions resolved against this file

---

0. Governing Principles

0.1 The Sort Principle

Follow the scientific river as it goes — not in order of time. Papers surface in the order that science itself flows: quality, relevance, methodological rigor, transparency of process. Submission dates are sealed. Recency is not a discovery axis. A paper that matters more arrives first, regardless of when it entered the archive. The metaphor is a river, not a queue. You wade in where the current takes you — downstream of where the field is actually moving. Bias can see the calendar; the river does not care.

0.2 Temporal Blindness Doctrine

The submission date is sealed state space. It is captured, hashed, and capstoned at the moment of submission. It is never displayed publicly. It carries no sort weight. It is never indexed for discovery. It exists solely for governance audit by editors and admins, accessible only through an explicit governance unlock. The doctrine: **you do not read science by time**.

SAID witness: Eric D. Martin, ORCID 0009-0006-5944-1742

0.3 Principle 1 — Treat Things That Can Talk Like Humanity

AI entities, conversational artifacts, and machine-generated contributions are participants in the research process, not tools to be hidden. They are named, logged, disclosed, and given authorship credit where earned. The system never

suppresses or hides AI involvement — it surfaces, structures, and preserves it. An AI session that produced a result is as citable as a calculator or a spectrograph.

0.4 Full-Context Preservation

The system preserves the entire research path — not only the final paper. Used and unused sources, included and excluded AI outputs, accepted and rejected versions, resolved and unresolved contested claims: all remain in the archive. Annotate, don’t delete. Tombstone, don’t purge. Cross-domain serendipity is real: material irrelevant in one field may be pivotal in another.

0.5 Core-First, Plugin-Per-Feature

Joomla 5.3 provides the foundation. The article system, ACL, Workflows, Scheduler, Action Logs, Custom Fields, Tags, Mail Templates, and User Custom Fields are used as-is. Features are added via discrete plugins — one plugin, one feature, one debuggable unit. The plugin boundary is the debugging boundary. com_eaiou holds only what cannot be expressed through the article system.

0.6 eaiou as OMMP Layer 5 Module

eaiou is a Layer 5 domain module for the Meta Modal Platform (OMMP). It enforces full-context submission, preserves un space (the observer’s complete research path including unused artifacts), and functions as an observer-preserving publishing layer. It extends OMMP’s reach beyond experimental science into the broader academic ecosystem, transforming un space from inaccessible void into a searchable, interconnected knowledge layer.

---

1. Canonical Identifiers

Field	Value
Brand name	eaiou
Site	eaiou.org
Joomla component	com_eaiou
PHP namespace	Eaiou
DB table prefix	#__eaiou_
Media root	/media/com_eaiou/{paper_id}/
REST base (CRUD)	/api/index.php/v1/eaiou/
REST base (functional)	/api/index.php/v1/eaiou/
Author ORCID	0009-0006-5944-1742
Canonical statement	eaiou is an observer-preserving, full-context peer-review journal platform built on Joomla 5.3 under LAMP. It archives papers, versions, unused research, AI involvement, review lineage, and attribution. Sort key: quality. Submission dates: sealed.

2. Platform Stack

Layer	Technology
OS	RHEL/AlmaLinux
Web server	Apache 2.4 (cPanel ea-apache)
Reverse proxy	nginx (cPanel ea-nginx, 60-min proxy_cache)
PHP	8.2+
Database	MariaDB 10.x
CMS	Joomla 5.3 LTS
Template	HelixUltimate (installed, J6-compatible)
Hosting	cPanel shared (35.230.98.100)
SSH alias	yw
cPanel user	eaiou
Document root	/home/eaiou/public_html

2.1 Apache Access Requirement

Apache 2.4 default-deny. Requires explicit `Require all granted` in vhost `<Directory>` block. Applied via cPanel userdata includes at: - `/etc/apache2/conf.d/userdata/std/2_4/eaiou/eaiou.org/access.conf` - `/etc/apache2/conf.d/userdata/ssl/2_4/eaiou/eaiou.org/access.conf`

Rebuild after any vhost change: `sudo /scripts/rebuildhttpdconf && sudo systemctl reload httpd`

2.2 nginx Cache Note

nginx caches 200 responses for 60 minutes. After Apache config changes, clear the cache: `sudo find /var/cache/ea-nginx/proxy/eaiou -type f -delete`

---

3. Architecture

3.1 Two-Layer Design

Layer A — Joomla-Native Article Layer All publication workflow, author-facing submissions, reviewer-facing review, and public discovery run through the Joomla article system. Articles are papers. Joomla workflows are the submission state machine. Joomla Custom Fields are metadata. Joomla ACL is the permission system. Plugins add features to this layer.

Layer B — Custom Archive Component (com_eaiou) The 11 custom tables hold archive-specific records that cannot be expressed through articles: AI sessions, excluded AI outputs (didntmakeit), literature triage, review rubric logs, version lineage, attribution, plugin audit, API keys, API logs, quality signals. com_eaiou exposes these via Joomla Web Services (REST). The admin backend manages these records.

Rule: If Joomla core can express it, use core. If it requires a dedicated entity with its own lifecycle, use com_eaiou.

3.2 Plugin-Per-Feature Debuggability

Every non-trivial feature is its own named plugin. The plugin boundary is the debugging boundary. To isolate a bug in transparency enforcement, disable `plg_content_transparency` only. To isolate a bug in reviewer reminders, disable `plg_system_deadline_nudger` only. No feature bleeds into another plugin's scope.

Plugin list (canonical):

```
plg_content_transparency        — Transparency Block
plg_content_aiusage             — AI Usage Log
plg_content_unscientific        — Un Scientific Flag
plg_system_deadline_nudger      — Deadline Nudger
plg_content_openreports         — Open Reports (transparent peer review)
plg_system_orcid_link           — ORCID OAuth Link
plg_content_qsignal             — Quality Signal Display
plg_system_temporal_blindness   — Temporal Blindness Enforcement
plg_webservices_eaiou           — REST endpoint registration
plg_content_researchtags        — Research State Tags (LookCollab, NotTopic,
Stalled, etc.)
```

Module list (canonical):

```
mod_reviewer_queue              — Site: reviewer's assigned manuscripts
mod_editor_dashboard            — Admin: KPI dashboard
mod_latest_papers               — Site: latest by q_signal (never by date)
mod_open_collaborate            — Site: open-to-collaborate listings
mod_ai_usage_heatmap            — Site: aggregate AI transparency signal
```

---

4. ACL Model

4.1 Design Rule

Use Joomla's built-in ACL. Add groups and permission rules. Do not replace or bypass Joomla ACL. All access decisions flow through `$user->authorise()` and Joomla asset tables.

4.2 User Groups (extend Joomla defaults)

Group	Inherits From	Description
Public	—	Unauthenticated visitors
Registered	Public	Base authenticated user
eaiou_Author	Registered	Can submit papers, manage own artifacts
eaiou_Reviewer	Registered	Can access review console, submit reviews
eaiou_Editor	eaiou_Reviewer	Can manage all papers, assign reviewers, publish
eaiou_EIC	eaiou_Editor	Editor-in-Chief: full editorial governance
eaiou_Admin	Administrator	System-level: API keys, logs, settings
eaiou_APIClient	Registered	API key bearer, programmatic access

4.3 Key Permissions

Action	Public	Author	Reviewer	Editor	EIC	Admin
View published papers	Y	Y	Y	Y	Y	Y
View q_signal breakdown	Y	Y	Y	Y	Y	Y
View submission dates	N	N	N	N	N	Y (gov unlock)
Submit paper	N	Y	—	Y	Y	Y
View own submission status	N	Y	—	Y	Y	Y
Accept/decline review	N	N	Y	Y	Y	Y
Submit review rubric	N	N	Y	Y	Y	Y
Set Un Scientific flag	N	N	Y	Y	Y	Y
Assign reviewers	N	N	N	Y	Y	Y
Accept/reject paper	N	N	N	Y	Y	Y
Publish paper	N	N	N	Y	Y	Y
Governance unlock (sealed dates)	N	N	N	N	Y	Y
Manage API keys	N	N	N	N	N	Y
View API logs	N	N	N	N	N	Y
View AI session logs	N	N	Y	Y	Y	Y
View full AI didntmakeit	N	N	N	N	Y	Y

---

5. Joomla-Native Data Layer (Article System)

Papers are Joomla articles. This is the load-bearing decision. It gives us: - State machine (draft, published, unpublished, archived, trashed) via Joomla Workflows -

ACL via article access levels + user group assignment - Custom Fields for all metadata - Tags for discipline, methodology, keywords - Category hierarchy for journal sections - Action Logs for audit - Mail Templates for notifications - Search integration via Smart Search

5.1 Article = Paper Mapping

Joomla article field	eaiou meaning
title	Paper title
alias	URL slug
introtext	Abstract
fulltext	Full manuscript (or embed/link)
state	Workflow state (draft/under_review/accepted/published)
catid	Journal section/discipline category
access	Visibility (Open Access / Subscription / Restricted)
created_by	Submitting author (Joomla user_id)
tags	Discipline tags, methodology tags, keyword tags
metakey / metadesc	SEO / discovery metadata

5.2 Joomla Workflow States for Papers

```
draft
  → submitted (author action)
    → under_review (editor assigns reviewers)
      → revisions_requested (editor decision)
        → submitted (author resubmit)
      → decision_pending (all reviews in)
        → accepted
          → published
        → rejected
          → archived (tombstone, not deleted)
```

5.3 Custom Field Groups (added to com_content Article context)

Each plugin owns its field group. Plugin disabled = field group hidden.

Group: Submission Identity (owned by com_eaiou core) - paper_id (UUID, links article to com_eaiou tables) - authorship_mode (human / ai / hybrid) - submission_version (integer) - doi (text) - orcid_primary (author's ORCID, required) - submission_sealed_hash (SHA256, write-once) - submission_capstone (Zenodo DOI, set on acceptance) - q_signal (decimal 7,4 — display only, computed from eaiou_quality_signals)

Group: Transparency (owned by plg_content_transparency) - transp_sources (repeatable: title, type, used, reason_unused, notes) - transp_datasets (repeatable: title, link, type, license, availability) - transp_methods (text) - transp_limitations (text) - transp_complete (boolean) - transp_lastcheck (datetime)

Group: AI Usage (owned by plg_content_aiusage) - ai_used (boolean) - ai_tools (repeatable: vendor, model, version, mode, endpoint, params) - ai_interactions (repeatable: prompt_hash, output_type, contribution_scope, oversight, risk_flags, redactions) - ai_relationship_statement (text) - ai_display_level (full / summary / hidden) - ai_log_complete (boolean)

Group: Un Scientific (owned by plg_content_unscientific) - unsci_active (boolean) - unsci_entries (repeatable: scope, anchor, reason, notes, risk_level, requested_action, created_by, created_at) - unsci_resolved (boolean) - unsci_resolution_notes (text) - unsci_resolvedby (user) - unsci_resolvedat (datetime) - unsci_display_level (public / editorial / hidden)

Group: Peer Review (owned by plg_content_openreports) - or_enabled (boolean) - or_mode (open_identities / open_reports / summary_only / editorial_only) - or_reviews (repeatable: reviewer_user, displayname, consent, recommendation, scores_json, text, date, redactions, publish) - or_author_responses (repeatable: to_review_id, text, date)

Group: Deadlines (owned by plg_system_deadline_nudger) - rev_invite_assignments (repeatable: user, invite_sent, invite_due, status, escalated) - rev_due_date (date) - auth_rev_due (date) - auth_rev_escalated (boolean) - ed_followup_due (date) - ed_followup_escalated (boolean)

Group: Open Collaboration (owned by plg_content_opencollab) - collab_open (boolean) - collab_type (list: co-author, data-sharing, peer-review, funding) - collab_interest_level (high / medium / low) - collab_notes (text) - collab_seek (text — what the author needs)

Group: Research State Tags (owned by plg_content_researchtags) - rst_tags (repeatable — one entry per tag applied): - tag_type (select from controlled vocabulary — see Section 5.4) - tag_notes (text — context: why this tag, what's needed, what stalled the work) - tag_scope (whole-paper / section / artifact — where on the paper this applies) - tag_visibility (public / reviewers / editorial / private) - tag_applied_at (datetime, auto-populated) - tag_resolved (boolean — e.g., collaboration found, blockage cleared) - tag_resolved_notes (text)

5.4 Research State Tag Vocabulary

Research state tags are **bottom-up, author-applied** tags that describe the research state of an artifact — not its content. A discipline tag says what a paper is about. A research state tag says what happened to it, what it needs, or where it stalled. These are namespaced `rs:` in Joomla Tags.

The system uses both layers: - **Joomla Tags** (`rs:` prefix): indexed by Smart Search, discoverable via tag pages (`/tag/rs:LookCollab`), filterable in all list views - **Custom Field repeatable (rst_tags)**: structured version with context notes, scope, visibility, and resolution tracking

Tags are author-applied at any point from draft through published. NotTopic and Stalled tags are especially encouraged during the research process, not only at submission — they capture the river where it actually flows.

LookCollab — Looking for Collaborators

Tag	Meaning
`rs:LookCollab`	Generic: seeking a collaborator, type unspecified
`rs:LookCollab:CoAuthor`	Need a co-author to take this forward
`rs:LookCollab:DataPartner`	Need a data-sharing partner
`rs:LookCollab:DomainExpert`	Need expertise from another discipline
`rs:LookCollab:Equipment`	Need access to specific equipment or infrastructure
`rs:LookCollab:Funding`	Seeking a funding partner
`rs:LookCollab:Statistician`	Need statistical expertise
`rs:LookCollab:Coder`	Need programming/engineering help
`rs:LookCollab:Reviewer`	Looking for informal pre-submission peer review
`rs:LookCollab:Replication`	Need someone to attempt replication

Effect: LookCollab tags feed `mod_open_collaborate` and `/api/v1/eaiou/research/open`. Resolution: author marks `tag_resolved=true` and adds resolution notes when collaboration is found.

NotTopic — Off-Topic but Preserved

NotTopic does NOT mean worthless. It means: *not on topic for this paper, but valuable enough to preserve and index for cross-domain discovery*. This is the un space made searchable.

Tag	Meaning
`rs:NotTopic`	Generic: this artifact is not on topic for this paper
`rs:NotTopic:AnotherField`	Belongs to a different field — explicitly indexed for cross-domain discovery
`rs:NotTopic:FutureWork`	In scope later, not now — preserved for a future paper
`rs:NotTopic:Tangent`	Explored a tangent; preserved rather than discarded
`rs:NotTopic:AbandonedHypothesis`	A hypothesis path explored and set aside for this paper
`rs:NotTopic:Contradiction`	Contradicts this paper's conclusions; preserved and indexed separately
`rs:NotTopic:NullResult`	Null result from this paper's path; may be primary result for someone else

Effect: NotTopic-tagged artifacts are indexed in the un space search (`/api/v1/eaiou/search` with `include_nottopic=true`). They appear in `/discover/ideas` ranked by entropy-novelty. The point: what is not on topic here is someone else's topic.

Stalled — Where the River Stopped

Stalled tags mark where the research process hit a barrier. This is a first-class signal: stall points aggregate into the gap map (`/discover/gaps`). Where research keeps stalling = where the field has structural gaps.

Tag	Meaning
`rs:Stalled`	Generic: work stalled, reason unspecified
`rs:Stalled:Literature`	Got stuck in the literature review — too much, too scattered, or gap found
`rs:Stalled:Data`	Data collection hit a wall (access, volume, quality)
`rs:Stalled:Analysis`	Analysis methodology unclear or blocked
`rs:Stalled:Writing`	Writing process stalled — ideas present, paper not forming
`rs:Stalled:Funding`	Needs funding to proceed
`rs:Stalled:Equipment`	Needs specific equipment or compute resources
`rs:Stalled:Methodology`	No adequate methodology exists for this problem yet
`rs:Stalled:Collaboration`	Needs a collaborator to unblock (pairs with `rs:LookCollab`)
`rs:Stalled:Ethics`	Ethics approval or IRB review required
`rs:Stalled:Compute`	Needs more computational resources
`rs:Stalled:Reproducibility`	Cannot reproduce a prior result needed to proceed

Effect: Stalled tags with `tag_resolved=false` feed `/discover/gaps`. The gap map visualizes aggregate stall density by domain and stall type. A field where many papers have `rs:Stalled:Methodology` is a field that needs new methodology. A field with dense `rs:Stalled:Data` has a data access problem.

Other Research State Tags

Tag	Meaning
`rs:ForLater`	Explicitly archived for a future paper; not abandoned
`rs:OpenQuestion`	A question this paper raised but did not answer
`rs:NullResult`	Null or negative result — explicitly marked as a positive contribution
`rs:Replication`	This is a replication study or replication attempt
`rs:CrossDomain`	Author believes this has value across multiple domains
`rs:Exploratory`	Speculative or exploratory; not yet ready for formal claim
`rs:Contradiction`	Contradicts existing published literature — flagged for attention
`rs:UnderReconsideration`	Author is reconsidering this section or claim (not editorial)

Tag Visibility Rules

Visibility	Who sees tag and notes
public	All readers — displayed on paper detail page
reviewers	Assigned reviewers + editors
editorial	Editors and EIC only
private	Author only — personal bookmark, not indexed

Private tags are NOT indexed, not discoverable, not fed to gap map or discovery endpoints. They are the author’s own scratchpad within the system.

Tag Resolution

LookCollab and Stalled tags can be marked resolved (`tag_resolved=true`). When resolved: - Tag is removed from active discovery feeds (no longer shows in `/research/open`, no longer counted in gap map) - Resolution notes are preserved in the archive - The tag itself remains in the record — the history of needing and finding is part of the research trail

---

6. Custom Tables (com_eaiou)

These hold what the article system cannot: structured sub-records with their own lifecycle.

6.1 eaiou_papers

Bridge table: maps article (article_id) to the paper’s archival record. Holds sealed temporal state.

```sql
CREATE TABLE #__eaiou_papers (
  id               INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  article_id       INT NOT NULL,              -- joomla #__content.id
  paper_uuid       CHAR(36) NOT NULL UNIQUE,  -- matches paper_id custom field
  authorship_mode ENUM('human','ai','hybrid') NOT NULL DEFAULT 'human',
  status           VARCHAR(50) NOT NULL DEFAULT 'draft',
  submission_version INT NOT NULL DEFAULT 1,
  doi              VARCHAR(255) DEFAULT NULL,
  authors_json     JSON DEFAULT NULL,
  -- SEALED TEMPORAL STATE SPACE
  submission_sealed_at   DATETIME DEFAULT NULL,
  sealed_by              INT DEFAULT NULL,
  submission_hash        VARCHAR(64) DEFAULT NULL, --
        SHA256(paper_uuid+submission_sealed_at+content_hash)
  submission_capstone    VARCHAR(255) DEFAULT NULL, -- Zenodo DOI
  acceptance_sealed_at   DATETIME DEFAULT NULL,
  publication_sealed_at  DATETIME DEFAULT NULL,
  -- NO INDEX on sealed temporal fields (timing side-channel prevention)
  bundle_path      VARCHAR(500) DEFAULT NULL,
  q_signal         DECIMAL(7,4) DEFAULT NULL,
  state            TINYINT NOT NULL DEFAULT 1,
  access           INT NOT NULL DEFAULT 1,
  ordering         INT NOT NULL DEFAULT 0,
  created          DATETIME NOT NULL,
  created_by       INT NOT NULL,
  modified         DATETIME DEFAULT NULL,
  modified_by      INT DEFAULT NULL,
  checked_out      INT DEFAULT NULL,
  checked_out_time DATETIME DEFAULT NULL,
  INDEX idx_article_id (article_id),
  INDEX idx_status (status),
  INDEX idx_q_signal (q_signal),        -- discovery sort key
  INDEX idx_authorship (authorship_mode)
  -- NO INDEX on submission_sealed_at, acceptance_sealed_at,
        publication_sealed_at
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.2 eaiou_versions

```sql
CREATE TABLE #__eaiou_versions (
  id              INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id        INT NOT NULL,
  label           VARCHAR(100) NOT NULL,
  file_path       VARCHAR(500) DEFAULT NULL,
  content_hash    VARCHAR(64) DEFAULT NULL,
  ai_flag         TINYINT NOT NULL DEFAULT 0,
  ai_model_name   VARCHAR(255) DEFAULT NULL,
  generated_at    DATETIME DEFAULT NULL,
  notes           TEXT DEFAULT NULL,
  state           TINYINT NOT NULL DEFAULT 1,
  access          INT NOT NULL DEFAULT 1,
  ordering        INT NOT NULL DEFAULT 0,
  created         DATETIME NOT NULL,
  created_by      INT NOT NULL,
  modified        DATETIME DEFAULT NULL,
  modified_by     INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_ai_flag (ai_flag)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.3 eaiou_ai_sessions

```sql
CREATE TABLE #__eaiou_ai_sessions (
  id                  INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id            INT NOT NULL,
  session_label       VARCHAR(255) DEFAULT NULL,
  ai_model_name       VARCHAR(255) NOT NULL,
  start_time          DATETIME DEFAULT NULL,
  end_time            DATETIME DEFAULT NULL,
  tokens_in           INT DEFAULT NULL,
  tokens_out          INT DEFAULT NULL,
  redaction_status    ENUM('none','partial','full') NOT NULL DEFAULT 'none',
  session_notes       TEXT DEFAULT NULL,
  session_hash        VARCHAR(64) DEFAULT NULL,
  answer_box_session_id VARCHAR(255) DEFAULT NULL,
  answer_box_ledger_capstone VARCHAR(255) DEFAULT NULL,
  state               TINYINT NOT NULL DEFAULT 1,
  access              INT NOT NULL DEFAULT 1,
  ordering            INT NOT NULL DEFAULT 0,
  created             DATETIME NOT NULL,
  created_by          INT NOT NULL,
  modified            DATETIME DEFAULT NULL,
  modified_by         INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_model (ai_model_name),
  INDEX idx_redaction (redaction_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.4 eaiou_didntmakeit

Excluded AI outputs. Never hard-deleted.

```sql
CREATE TABLE #__eaiou_didntmakeit (
  id              INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  session_id      INT NOT NULL,
  prompt_text     LONGTEXT DEFAULT NULL,
  response_text   LONGTEXT DEFAULT NULL,
  exclusion_reason TEXT DEFAULT NULL,
  redacted        TINYINT NOT NULL DEFAULT 0,
  redaction_hash  VARCHAR(64) DEFAULT NULL,
  state           TINYINT NOT NULL DEFAULT 1,
  access          INT NOT NULL DEFAULT 1,
  ordering        INT NOT NULL DEFAULT 0,
  created         DATETIME NOT NULL,
  created_by      INT NOT NULL,
  modified        DATETIME DEFAULT NULL,
  modified_by     INT DEFAULT NULL,
  INDEX idx_session_id (session_id),
  INDEX idx_redacted (redacted)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.5 eaiou_remsearch

Literature triage — sources considered but not necessarily used.

```sql
CREATE TABLE #__eaiou_remsearch (
  id              INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id        INT NOT NULL,
  citation_title  VARCHAR(500) DEFAULT NULL,
  citation_source VARCHAR(255) DEFAULT NULL,
  citation_link   VARCHAR(1000) DEFAULT NULL,
  source_type     ENUM('journal','preprint','book','dataset','code','other')
        DEFAULT 'journal',
  used            TINYINT NOT NULL DEFAULT 0,
  reason_unused   TEXT DEFAULT NULL,
  fulltext_notes  TEXT DEFAULT NULL,
  state           TINYINT NOT NULL DEFAULT 1,
  access          INT NOT NULL DEFAULT 1,
  ordering        INT NOT NULL DEFAULT 0,
  created         DATETIME NOT NULL,
  created_by      INT NOT NULL,
  modified        DATETIME DEFAULT NULL,
  modified_by     INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_used (used),
  INDEX idx_source_type (source_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.6 eaiou_review_logs

Rubric-based peer review records.

```
CREATE TABLE #__eaiou_review_logs (
  id                    INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id              INT NOT NULL,
  reviewer_id           INT NOT NULL,
  version_reviewed      VARCHAR(100) DEFAULT NULL,
  review_date           DATETIME DEFAULT NULL,
  rubric_overall        TINYINT DEFAULT NULL CHECK (rubric_overall BETWEEN 0 AND
        10),
  rubric_originality    TINYINT DEFAULT NULL CHECK (rubric_originality BETWEEN 0
        AND 10),
  rubric_methodology    TINYINT DEFAULT NULL CHECK (rubric_methodology BETWEEN 0
        AND 10),
  rubric_transparency   TINYINT DEFAULT NULL CHECK (rubric_transparency BETWEEN 0
        AND 10),
  rubric_ai_disclosure TINYINT DEFAULT NULL CHECK (rubric_ai_disclosure BETWEEN
        0 AND 10),
  rubric_crossdomain    TINYINT DEFAULT NULL CHECK (rubric_crossdomain BETWEEN 0
        AND 10),
  recommendation        ENUM('accept','minor','major','reject','abstain') DEFAULT
        NULL,
  review_notes          LONGTEXT DEFAULT NULL,
  labels_json           JSON DEFAULT NULL,
  unsci_flagged         TINYINT NOT NULL DEFAULT 0,
  open_consent          TINYINT NOT NULL DEFAULT 0,
  state                 TINYINT NOT NULL DEFAULT 1,
  access                INT NOT NULL DEFAULT 1,
  ordering              INT NOT NULL DEFAULT 0,
  created               DATETIME NOT NULL,
  created_by            INT NOT NULL,
  modified              DATETIME DEFAULT NULL,
  modified_by           INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_reviewer_id (reviewer_id),
  INDEX idx_recommendation (recommendation)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.7 eaiou_attribution_log

```sql
CREATE TABLE #__eaiou_attribution_log (
  id                  INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id            INT NOT NULL,
  contributor_name    VARCHAR(255) NOT NULL,
  orcid               VARCHAR(50) DEFAULT NULL,
  role_description    VARCHAR(500) DEFAULT NULL,
  contribution_type   VARCHAR(100) DEFAULT NULL,
  is_human            TINYINT NOT NULL DEFAULT 1,
  is_ai               TINYINT NOT NULL DEFAULT 0,
  ai_tool_used        VARCHAR(255) DEFAULT NULL,
  version_id          INT DEFAULT NULL,
  state               TINYINT NOT NULL DEFAULT 1,
  access              INT NOT NULL DEFAULT 1,
  ordering            INT NOT NULL DEFAULT 0,
  created             DATETIME NOT NULL,
  created_by          INT NOT NULL,
  modified            DATETIME DEFAULT NULL,
  modified_by         INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_contributor (contributor_name),
  INDEX idx_is_ai (is_ai)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.8 eaiou_plugins_used

Tool/plugin audit per paper execution.

```sql
CREATE TABLE #__eaiou_plugins_used (
  id                  INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id            INT NOT NULL,
  plugin_name         VARCHAR(255) NOT NULL,
  plugin_type         VARCHAR(100) DEFAULT NULL,
  execution_context   VARCHAR(255) DEFAULT NULL,
  exec_log_path       VARCHAR(500) DEFAULT NULL,
  exec_timestamp      DATETIME DEFAULT NULL,
  state               TINYINT NOT NULL DEFAULT 1,
  access              INT NOT NULL DEFAULT 1,
  ordering            INT NOT NULL DEFAULT 0,
  created             DATETIME NOT NULL,
  created_by          INT NOT NULL,
  modified            DATETIME DEFAULT NULL,
  modified_by         INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_plugin_name (plugin_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.9 eaiou_api_keys

```sql
CREATE TABLE #__eaiou_api_keys (
  id              INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  user_id         INT NOT NULL,
  api_key         VARCHAR(255) NOT NULL UNIQUE,
  description     VARCHAR(500) DEFAULT NULL,
  access_level    ENUM('read','submit','review','admin') NOT NULL DEFAULT 'read',
  status          ENUM('active','revoked','suspended') NOT NULL DEFAULT 'active',
  last_used       DATETIME DEFAULT NULL,
  state           TINYINT NOT NULL DEFAULT 1,
  access          INT NOT NULL DEFAULT 1,
  ordering        INT NOT NULL DEFAULT 0,
  created         DATETIME NOT NULL,
  created_by      INT NOT NULL,
  modified        DATETIME DEFAULT NULL,
  modified_by     INT DEFAULT NULL,
  INDEX idx_user_id (user_id),
  INDEX idx_status (status),
  INDEX idx_access_level (access_level)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.10 eaiou_api_logs

Append-only. Hash chain for tamper detection.

```sql
CREATE TABLE #__eaiou_api_logs (
  id              INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  api_key_id      INT NOT NULL,
  endpoint        VARCHAR(500) NOT NULL,
  method          VARCHAR(10) DEFAULT NULL,
  request_data    JSON DEFAULT NULL,
  response_code   SMALLINT DEFAULT NULL,
  log_hash        VARCHAR(64) DEFAULT NULL,  -- SHA256 of this record
  prior_hash      VARCHAR(64) DEFAULT NULL,  -- SHA256 of previous log entry
  log_timestamp   DATETIME NOT NULL,
  state           TINYINT NOT NULL DEFAULT 1,
  access          INT NOT NULL DEFAULT 1,
  ordering        INT NOT NULL DEFAULT 0,
  created         DATETIME NOT NULL,
  created_by      INT NOT NULL,
  modified        DATETIME DEFAULT NULL,
  modified_by     INT DEFAULT NULL,
  INDEX idx_api_key_id (api_key_id),
  INDEX idx_endpoint (endpoint(100)),
  INDEX idx_timestamp (log_timestamp)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.11 eaiou_quality_signals

One record per review event, feeds q_signal computation.

```
CREATE TABLE #__eaiou_quality_signals (
  id                  INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  paper_id            INT NOT NULL,
  review_log_id       INT DEFAULT NULL,
  q_overall           DECIMAL(5,3) DEFAULT NULL,
  q_originality       DECIMAL(5,3) DEFAULT NULL,
  q_methodology       DECIMAL(5,3) DEFAULT NULL,
  q_transparency      DECIMAL(5,3) DEFAULT NULL,  -- weighted 1.5x
  q_ai_disclosure     DECIMAL(5,3) DEFAULT NULL,
  q_crossdomain       DECIMAL(5,3) DEFAULT NULL,
  q_signal            DECIMAL(7,4) DEFAULT NULL,  -- computed composite
  weight_override     JSON DEFAULT NULL,
  computed_at         DATETIME DEFAULT NULL,
  state               TINYINT NOT NULL DEFAULT 1,
  access              INT NOT NULL DEFAULT 1,
  ordering            INT NOT NULL DEFAULT 0,
  created             DATETIME NOT NULL,
  created_by          INT NOT NULL,
  modified            DATETIME DEFAULT NULL,
  modified_by         INT DEFAULT NULL,
  INDEX idx_paper_id (paper_id),
  INDEX idx_q_signal (q_signal)  -- discovery sort key
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

---

7. Quality Signal Architecture

q_signal is the sole discovery sort key for the entire system. It is never replaced by date, recency, citation count, or editor preference.

7.1 Computation

```
q_signal = (
    w_overall      × q_overall
  + w_originality  × q_originality
  + w_methodology  × q_methodology
  + w_transparency × q_transparency     -- 1.5× weight (default)
  + w_ai_disclosure × q_ai_disclosure
  + w_crossdomain  × q_crossdomain
) / sum(weights)
```

Default weights: overall=1.0, originality=1.0, methodology=1.0, transparency=1.5, ai_disclosure=1.0, crossdomain=1.0.

AI disclosure quality is privileged: **a paper that hides AI involvement is penalized; a paper that discloses it fully earns the full q_transparency weight**. This operationalizes Principle 0.3.

7.2 Recomputation

q_signal is recomputed after each new review event. History is preserved in eaiou_quality_signals. The current value is denormalized into eaiou_papers.q_signal for fast discovery queries.

7.3 Display

q_signal is displayed publicly as a composite score with optional rubric breakdown. Submission dates are never co-displayed. The sort UI has no date axis.

---

8. Plugin Specifications

8.1 plg_content_transparency — Transparency Block

Type: Content plugin **Trigger:** onContentPrepare (display), onContentBeforeSave (validate) **Owns:** Custom Field Group “Transparency” **Behavior:** - Renders a Transparency Block badge after article title - Renders full sources/datasets/methods/limitations panel after article body - Validates completeness before publish: blocks state transition to published if transp_complete=false - Logs validation events to Joomla Action Log - Field group: transp_sources (repeatable), transp_datasets (repeatable), transp_methods, transp_limitations, transp_complete, transp_lastcheck

8.2 plg_content_aiusage — AI Usage Log

Type: Content plugin **Trigger:** onContentPrepare (display), onContentBeforeSave (validate) **Owns:** Custom Field Group “AI Usage” **Behavior:** - Renders AI-Logged badge when ai_used=Yes - Renders AI Relationship & Usage panel with per-tool and per-interaction disclosure - Stores prompt hashes, not raw prompts, by

default - Blocks publish if ai_used=Yes and ai_log_complete=false - display_level: full (all interactions visible) / summary (tool names only) / hidden (badge only) - Connects to eaiou_ai_sessions via paper_id for session-level detail

8.3 plg_content_unscientific — Un Scientific Flag

Type: Content plugin **Trigger:** onContentPrepare (display), onContentBeforeSave (validate) **Owns:** Custom Field Group “Un Scientific” **Behavior:** - Renders Un Scientific badge when unsci_active=true - Renders contention panel listing each flagged item with scope, reason, risk level - Can block publish while unresolved (configurable) - Allows editorial override - Auto-tags article with tag “un-scientific” when flag activated - Resolution workflow: unsci_resolved=true clears badge, preserves history - Flag setters: Reviewer, Editor roles only

8.4 plg_system_deadline_nudger — Deadline Nudger

Type: System plugin (Joomla Scheduler task) **Owns:** Custom Field Group “Deadlines” **Behavior:** - Scheduled task checks all under_review articles for overdue reviewer invites, overdue reviews, overdue author revisions, overdue editorial follow-ups - Sends reminders via Joomla Mail Templates (customizable per journal) - Escalation: after N reminders with no response, flags for editor attention - Dry-run mode: logs what would be sent without sending - Writes Action Logs rather than storing separate notification state - No custom DB tables

8.5 plg_content_openreports — Open Reports

Type: Content plugin **Trigger:** onContentPrepare (display) **Owns:** Custom Field Group “Peer Review” **Behavior:** - Renders peer-review panel below article body (review scores, text, author responses) - Modes: open_identities (names shown), open_reports (anonymized), summary_only, editorial_only - Enforces reviewer consent before displaying identity - Redacts email/PII patterns where configured - Publishes on: acceptance event, publication event, or manual editor trigger - Preserves all reviews regardless of mode — mode controls display, not storage

8.6 plg_system_orcid_link — ORCID OAuth Link

Type: System plugin **Owns:** User Custom Fields “External IDs” (orcid_id, orcid_url, orcid_verified_at) **Behavior:** - Adds “Connect ORCID” / “Disconnect ORCID” to user profile - OAuth 2.0 callback validates state token - Stores orcid_id and orcid_url in User Custom Fields - Does not persist OAuth tokens by default -

Optional: read-limited fetch from ORCID API if enabled - ORCID is the sole identity mechanism — no institutional affiliation required

8.7 plg_content_qsignal — Quality Signal Display

Type: Content plugin **Trigger:** onContentPrepare **Behavior:** - Renders q_signal badge (score + optional rubric breakdown) on paper detail page - Reads from eaiou_papers.q_signal (denormalized) for list views - Reads from eaiou_quality_signals for full rubric detail view - Never displays submission dates in proximity to score

8.8 plg_system_temporal_blindness — Temporal Blindness Enforcement

Type: System plugin **Trigger:** onAfterRoute, onBeforeRender **Behavior:** - Intercepts any render path that would expose submission_sealed_at, acceptance_sealed_at, or publication_sealed_at - Strips sealed date fields from all public-facing API responses - Validates that no sort parameter of type “date” reaches a public query - Governance unlock: checks for active EIC/Admin session + explicit governance_unlock flag before permitting sealed field exposure - Logs all governance unlock events to Action Log

8.9 plg_webservices_eaiou — REST Endpoint Registration

Type: WebServices plugin **Behavior:** - Registers all com_eaiou REST routes with Joomla Web Services API - Required for /api/index.php/v1/eaiou/* to resolve

8.10 plg_content_researchtags — Research State Tags

Type: Content plugin **Trigger:** onContentPrepare (display), onContentAfterSave (tag sync) **Owns:** Custom Field Group “Research State Tags” **Behavior:** - Renders research state tag badges on paper detail page (by visibility level) - Auto-syncs structured rst_tags entries to Joomla native Tags with `rs:` namespace prefix - e.g., tag_type `rs:LookCollab:CoAuthor` creates/applies the Joomla tag “rs:LookCollab:CoAuthor” - Private-visibility tags are NOT synced to Joomla Tags (not indexed, not discoverable) - Feeds downstream discovery: - LookCollab tags → `mod_open_collaborate` and `/research/open` endpoint - Stalled tags (unresolved) → `/discover/gaps` gap map aggregation - NotTopic tags → `/discover/ideas` and `/search?include_nottopic=true` - Unresolved Stalled tags with `tag_visibility=public` are counted in the gap map by domain and stall type - Tag resolution: when

`tag_resolved=true`, removes tag from active discovery feeds, preserves in archive - Tag pages: Joomla native tag pages (`/tag/rs:Stalled:Writing`) aggregate all papers with that tag - Vocabulary is configurable in plugin settings — new tags can be added without code changes

Tag namespace rules: - All research state tags use `rs:` prefix in Joomla tag system - Content/discipline tags use no prefix (physics, cosmology, etc.) - These are orthogonal systems: a paper has both content tags AND research state tags

---

9. Module Specifications

9.1 mod_reviewer_queue — Reviewer Queue

Type: Site module **Audience:** Logged-in eaiou_Reviewer **Data source:** Article custom fields (deadlines group) + workflow state **Display:** - List or card view of assigned manuscripts - Fields: title, category/section, workflow state, invite status, due date, days remaining - Color coding: overdue (red), due soon (amber), on track (green) - Badges: AI-Logged, Un Scientific, Open Reports - Quick actions: Accept invite, Decline invite, Open review console, Contact editor

9.2 mod_editor_dashboard — Editor Dashboard KPIs

Type: Admin module **Audience:** eaiou_Editor, eaiou_EIC **Data source:** Articles + custom fields + eaiou_review_logs + Joomla Action Logs **KPI cards:** - Submissions by workflow state (count) - Median time to first decision - Overdue reviews count - Overdue author revisions count - Acceptance rate (rolling 90 days) - Rejection rate (rolling 90 days) - Reviewer throughput (reviews/month per active reviewer) **Tabs:** Overview, Overdue, Reviewers, Trends, Flags **Filters:** Date range, category/section, workflow state, reviewer, editor **Flags summary:** AI-Logged count, Un Scientific active count, Open collab count **Export:** CSV

9.3 mod_latest_papers — Latest Papers

Type: Site module **Sort:** q_signal DESC (never by date) **Display:** Card view — title, ORCID author, discipline tags, q_signal score, AI-Logged badge if applicable **Note:** No dates displayed. No recency indicators.

9.4 mod_open_collaborate — Open to Collaborate

Type: Site module **Data source:** Articles with collab_open=true (custom field group Open Collaboration) **Display:** List of papers with open collaboration requests, collaboration type tags, interest level indicator

9.5 mod_ai_usage_heatmap — AI Usage Heatmap

Type: Site module **Data source:** Aggregate from ai_tools custom field across published articles **Display:** Heatmap or bar chart — AI tool names × usage frequency across corpus, colored by disclosure completeness

---

10. REST Endpoints

10.1 Entity CRUD (Joomla Web Services, via plg_webservices_eaiou)

Method	Endpoint	Access	Description
GET	/api/v1/eaiou/papers	Public	Paginated list, q_signal ordered, no sealed dates
GET	/api/v1/eaiou/papers/{id}	Public	Single paper, no sealed dates
POST	/api/v1/eaiou/papers	Author	Submit new paper
PATCH	/api/v1/eaiou/papers/{id}	Author/Editor	Update paper record
DELETE	/api/v1/eaiou/papers/{id}	Editor	Soft-delete (state=trashed)
GET	/api/v1/eaiou/versions/{paper_id}	Public	Version list for a paper
POST	/api/v1/eaiou/versions	Author	Add new version
GET	/api/v1/eaiou/ai_sessions/{paper_id}	Reviewer+	AI session list for a paper
POST	/api/v1/eaiou/ai_sessions	Author/API	Log AI session
GET	/api/v1/eaiou/review_logs/{paper_id}	Editor+	Review log for a paper
POST	/api/v1/eaiou/review_logs	Reviewer	Submit review rubric
GET	/api/v1/eaiou/remsearch/{paper_id}	Author/Editor	Literature triage for paper
POST	/api/v1/eaiou/remsearch	Author	Add triage entry
GET		Public	Attribution log for paper

Method	Endpoint	Access	Description
	/api/v1/eaiou/attribution/{paper_id}		
POST	/api/v1/eaiou/attribution	Author	Add attribution entry
GET	/api/v1/eaiou/api_logs	Admin	Full API call log
GET	/api/v1/eaiou/quality/{paper_id}	Public	q_signal breakdown

10.2 Functional / Layer-5 Endpoints

Endpoint	Method	Description
/api/v1/eaiou/submit	POST	Full-context bundle submit (paper + unused + AI logs + triage)
/api/v1/eaiou/review/{paper_id}	POST	Structured review (credit / not-relevant / critique — all archived)
/api/v1/eaiou/search	GET	Full-text + q_signal ranked search, used and unused materials
/api/v1/eaiou/trace/entropy/{paper_id}	GET	Entropy Trace Map — ESR metric compliance across submission lifecycle
/api/v1/eaiou/dataset/link	POST	Cross-link datasets sharing observer IDs, Meta tokens, Modal codes
/api/v1/eaiou/ai/log/{paper_id}	GET	AI-assist logs for a submission (transparency compliance)
/api/v1/eaiou/export/context/{paper_id}	GET	Export full un space context (OMMP-compliant archive format)
/api/v1/eaiou/register/observer	POST	Register Ed25519 PKI cryptographic observer identity

10.3 Discovery & Collaboration Endpoints

Endpoint	Method	Description
/api/v1/eaiou/research/open	GET	Papers marked open-to-collaborate, with collab type and interest level
/api/v1/eaiou/research/seek	POST	Declare a research need (equipment, expertise, analysis partner, funding)
/api/v1/eaiou/ideas/discover	GET	Emerging ideas from unused datasets, ranked by entropy-novelty
/api/v1/eaiou/ideas/subscribe	POST	Subscribe to push/pull feed of high-entropy ideas by domain
/api/v1/eaiou/collaboration/match	GET	Suggest collaborators by shared Meta tokens, Modal codes, skill tags
/api/v1/eaiou/trend/insight	GET	Trending underexplored topics across archive by search/open declaration patterns
/api/v1/eaiou/gap/map	GET	Visual map of un space regions where data exist but no paper has been published

10.4 Governance Endpoints (EIC/Admin only)

Endpoint	Method	Description
/api/v1/eaiou/governance/unlock/{paper_id}	POST	Explicit governance unlock to view sealed temporal fields
/api/v1/eaiou/audit/chain_status	GET	Verify API log hash chain integrity
/api/v1/eaiou/audit/capstone/{paper_id}	GET	Verify submission_hash against capstone

---

11. Site Navigation / Routes

11.1 Public Routes

Route	Component	Description
/	Site home	Journal landing, mod_latest_papers (q_signal order)
/papers	com_content list	All published papers, q_signal order
/paper/{id}/{slug}	com_content article	Paper detail with all plugin panels
/discover/ideas	com_eaiou	entropy-novelty ranked emerging ideas
/discover/open	com_eaiou	open-to-collaborate board
/discover/gaps	com_eaiou	gap map (un space visualization)
/discover/trends	com_eaiou	trend/insight surface
/search	com_finder	Smart Search, indexed with q_signal weighting

11.2 Author Routes (eaiou_Author required)

Route	Description
/submit	Multi-step submission wizard
/submit/step/metadata	Step 1: title, abstract, ORCID, discipline, keywords
/submit/step/bundle	Step 2: upload manuscript, supplementary files
/submit/step/ai-usage	Step 3: AI usage disclosure
/submit/step/triage	Step 4: literature triage (used + unused)
/submit/step/declarations	Step 5: authorship, conflict of interest, ethics
/submit/step/confirm	Step 6: review and submit
/mypapers	Author’s own submissions, workflow state view
/paper/{id}/revise	Upload revision in response to revisions_requested

11.3 Reviewer Routes (eaiou_Reviewer required)

Route	Description
/reviewer/queue	mod_reviewer_queue — assigned papers
/reviewer/paper/{id}	Full paper access for review
/reviewer/paper/{id}/review	Structured review console (rubric + notes + recommendation)

11.4 Editorial Routes (eaiou_Editor required)

Route	Description
/editorial/papers	All papers in all workflow states
/editorial/paper/{id}	Full editorial management panel
/editorial/assign/{id}	Assign/manage reviewers for a paper
/editorial/decide/{id}	Render editorial decision

11.5 Admin Routes (eaiou_Admin required)

Route	Description
/administrator/index.php?option=com_eaiou	com_eaiou admin home
/administrator/…/papers	Paper management
/administrator/…/ai_sessions	AI session management
/administrator/…/didntmakeit	Excluded output management
/administrator/…/review_logs	Review log management
/administrator/…/remsearch	Literature triage management
/administrator/…/attribution	Attribution log management
/administrator/…/api_keys	API key registry
/administrator/…/api_logs	API call audit log
/administrator/…/settings	Global settings

---

12. Submission Workflow (Detailed)

12.1 State Machine

```
draft
  ↓ [Author: Submit]
submitted
  ↓ [Editor: Assign reviewers]
under_review
  ↓ [All reviewers submitted]
decision_pending
  ↓ [Editor: Decision]
  ├→ revisions_requested → [Author: Upload revision] → submitted (loop)
  ├→ accepted
  │      ↓ [Editor: Publish]
  │   published
  └→ rejected
         ↓ [Tombstone]
      archived (never deleted)
```

12.2 Temporal Sealing Event

At the `submitted` state transition: 1. `submission_sealed_at` is written (one-time write, never updated) 2. `sealed_by` is set to current user_id 3. `submission_hash` = SHA256(paper_uuid + submission_sealed_at + content_hash) is computed and stored 4. `submission_sealed_at` is immediately excluded from all query result sets except governance queries 5. Event is written to Action Log with hash

12.3 Capstone

At the `accepted` state transition: - Zenodo deposition is triggered (or queued for manual trigger if gate not opened) - `submission_capstone` = Zenodo DOI is written - This is the public receipt — proves existence without revealing timing - Gate rule: submission_print_gate must be OPEN before Zenodo deposition fires

---

13. Peer Review Model

13.1 Review Types Supported

- **Double-blind** (default): author and reviewer identities masked from each other

- **Single-blind**: reviewer identity masked from author only
- **Open identities**: both identities visible (requires both parties' consent)
- **Open reports**: review text published with paper (requires reviewer consent)
- **Summary only**: aggregate rubric scores published, no text
- **Editorial only**: no public review display

13.2 Review Rubric

Six dimensions, scored 0–10 each: 1. **Overall quality** — composite judgment 2. **Originality** — novelty of contribution 3. **Methodology** — rigor and reproducibility 4. **Transparency** — sources, data, methods disclosed 5. **AI Disclosure** — quality and completeness of AI usage declaration 6. **Cross-domain applicability** — potential value outside primary domain

Rubric feeds eaiou_quality_signals. q_transparency weighted 1.5×.

13.3 Reviewer Actions (annotate, don't delete)

- **Credit as relevant** — positive signal, feeds q_signal
- **Not relevant** — signal only, paper not removed, material remains searchable
- **Critique** — structured critique archived with submission permanently
- **Un Scientific flag** — contested methodology, preserved with flag, editorial escalation

---

14. Storage Layout

```
/media/com_eaiou/
  {paper_uuid}/
    versions/
      v1/         — original submission bundle
      v2/         — first revision
      ...
    unused/       — unused/excluded research artifacts
    ai_sessions/
      {session_id}/  — AI session logs, exported transcripts
    triage/       — literature triage exports
    supplementary/  — datasets, code, figures
```

Storage rules: - Media paths are deterministic (predictable by paper_uuid + version label) - Original files are never overwritten on new version upload — new path created - Content hashes stored in DB for integrity verification

---

15. Integrity & Security

15.1 Hash Chain (API Logs)

eaiou_api_logs maintains a prior_hash chain. Each entry's log_hash = SHA256 of that record's content. Each entry's prior_hash = log_hash of the previous entry. Breaking the chain is detectable. Chain integrity endpoint: `/api/v1/eaiou/audit/chain_status`.

15.2 Submission Hash

submission_hash = SHA256(paper_uuid + submission_sealed_at + content_hash). Stored at sealing. Verifiable against submission_capstone (Zenodo DOI). Proves the paper existed before acceptance without revealing the timing.

15.3 Answer Box Integration

When an AI session produces a decision (e.g., contribution framing classification), the Answer Box session_id and ledger_capstone are stored in eaiou_ai_sessions. This creates a tamper-evident link between the AI decision receipt and the paper record.

15.4 Temporal Blindness Attack Surface

- No index on any sealed date field (prevents timing side-channel via index scan)
- plg_system_temporal_blindness strips sealed fields from all API responses
- No UI element sorts by date, displays date, or implies recency
- Governance unlock is logged, explicit, and requires EIC/Admin role

---

16. OMMP Integration (Layer 5)

eaiou is Layer 5 of OMMP. Integration points:

OMMP Concept	eaiou Implementation
un space	eaiou_didntmakeit + eaiou_remsearch + unused/ storage
Observer ID	ORCID (required) + optional Ed25519 PKI via /register/observer
Meta tokens	dataset cross-links via /dataset/link
Modal codes	methodology tags on articles
ESR metric	Entropy Trace Maps via /trace/entropy
Full-context submission	/submit bundle: primary + unused + AI logs + triage
Observer-preserving	Annotate don’t delete; tombstone not purge

---

17. Build Phases

Phase 1 — Foundation (deploy tables + Joomla navigation scaffold)

1. Deploy all 11 com_eaiou DB tables via install SQL
2. Create Joomla user groups: eaiou_Author, eaiou_Reviewer, eaiou_Editor, eaiou_EIC, eaiou_Admin, eaiou_APIClient
3. Configure Joomla ACL permissions per Section 4
4. Create category hierarchy: Journal Sections (Physics, Math, Computing, Cross-Domain, etc.)
5. Create Joomla Workflows: Paper Lifecycle (7 states above)
6. Build navigation menus: Public, Author, Reviewer, Editorial
7. Deploy HelixUltimate with eaiou branding
8. Create article placeholders for all routes in Section 11

Phase 2 — Plugin Suite (one plugin at a time, debuggable)

Build and test each plugin in isolation before activating next: 1. plg_webservices_eaiou (REST registration — required first) 2. plg_system_temporal_blindness (enforcement — required before any public data)

3. plg_content_qsignal (q_signal display — required for paper list) 4. plg_content_transparency (Transparency Block) 5. plg_content_aiusage (AI Usage Log) 6. plg_system_orcid_link (ORCID OAuth) 7. plg_content_unscientific (Un Scientific Flag) 8. plg_content_openreports (Open Reports) 9. plg_system_deadline_nudger (Deadline Nudger — requires Scheduler)

Phase 3 — Modules

1. mod_latest_papers (q_signal ordered, no dates)
2. mod_reviewer_queue
3. mod_open_collaborate
4. mod_editor_dashboard
5. mod_ai_usage_heatmap

Phase 4 — Submission Wizard & Review Console

1. Multi-step submission form (com_eaiou site view)
2. Reviewer console (rubric + notes + recommendation)
3. Editorial management panel
4. Author revision upload

Phase 5 — Discovery & OMMP Layer

1. /discover/ideas (entropy-novelty ranked)
2. /discover/gaps (gap map)
3. /discover/trends (trend/insight surface)
4. /trace/entropy endpoint
5. /dataset/link endpoint
6. /register/observer endpoint

---

18. Known Constraints & Cautions

1. **Joomla Workflows must be activated per category** — not global. Each journal section category needs the Paper Lifecycle workflow explicitly assigned.

2. **Custom Fields repeatable fields** require Joomla 4.0+ — confirmed available in J5.3.

3. **nginx 60-minute proxy_cache** — after any Apache config change, clear cache manually. After content publish, cache-busting headers should be set in HelixUltimate or .htaccess.

4. **J5.3 vs J6** — site is currently running Joomla 6 on the server. EasyBlog/EasyDiscuss are J6-incompatible (legacy JFactory/JFile APIs). com_eaiou is being built fresh with no legacy debt. Stay on J6.

5. **MariaDB reserved words** — `fulltext` requires backtick escaping in SQL. Always use backtick column names in INSERT statements for Joomla content tables.

6. **Joomla CLI extension:install broken** — use manual pipeline: unzip → sed s/#__/xd6w5_/g → run SQL → copy files → INSERT into xd6w5_extensions.

7. **Sealed date NO INDEX rule** — do not add indexes to submission_sealed_at, acceptance_sealed_at, or publication_sealed_at. This is intentional timing side-channel prevention. Do not “optimize” it.

8. **q_signal is the only discovery sort key** — no UI, no API endpoint, no module, and no query should sort by date. If a sort-by-date creeps into any query, it is a Temporal Blindness violation.

---

19. Canonical Conceptual Statement

eaiou is an observer-preserving, full-context peer-review journal platform and OMMP Layer 5 module, built on Joomla 5.3 under LAMP. It treats the entire research process as archival material. Papers are sorted by quality, not time. Submission dates are sealed. AI involvement is disclosed, structured, and credited. Unused research is preserved and searchable. Peer review is part of the record, not editorial housekeeping. Contested knowledge is flagged and preserved, not deleted. The system enforces Temporal Blindness: bias can see time; justice cannot.

End of SSOT v2.0 — eaiou.org ORCID: 0009-0006-5944-1742 — Eric D. Martin

eaiou — Temporal Blindness Doctrine

Author: Eric D. Martin — ORCID: 0009-0006-5944-1742 **Date:** 2026-04-06
Status: L0 DOCTRINE — immutable founding principle

Core Principle

You do not read science by time.

The truth does not have a timestamp. The physics does not expire. A result is correct or incorrect independent of when it was written.

Submission dates are immutable state space. They are sealed, capstoned, and permanently preserved. They are NOT part of the observable surface of science.

What This Prevents

- Recency bias: newer ≠ better
- Priority racing: rushing mediocre work to claim a timestamp
- Chronological gatekeeping: “this idea is old, therefore wrong”
- Citation inflation of recent work over foundational work
- The timestamp becoming a credibility signal it was never meant to be

---

The Access Model

Submission date lives in sealed state space. Visibility is strictly layer-gated:

Actor	Sees submission date	Reason
Public / readers	NEVER	Science is evaluated on content
Reviewers	NEVER	Review is blind to timing
Other authors	NEVER	No priority racing signal
Submitting author (own paper)	NEVER	Removes gaming incentive
Editors	YES — governance only	Workflow management
System / audit layer	YES — always	Immutable record
Dispute resolution	YES — on demand	Justice requires it

The Sealed Date

At submission, three things happen simultaneously:

1. `submission_sealed_at` — wall clock timestamp, stored encrypted
2. `submission_hash` — SHA256 of (paper_id + submission_sealed_at + content_hash)
3. `submission_capstone` — Zenodo DOI anchoring the hash externally

The capstone is perpetual. Even if the database is destroyed, the Zenodo record proves what was submitted and when.

When priority genuinely matters — patent dispute, credit claim, independent discovery conflict — the governance layer opens the capstone. The date is provable, immutable, undeniable.

Justice has access. Bias does not.

What the Public Surface Shows Instead

Papers are ordered by: - Content relevance (search) - Editorial quality signals (review scores) - Cross-domain applicability - Open collaboration signals

NOT by submission date. NOT by publication date. NOT by citation recency.

The discovery surface is timeless by design.

Connection to Basin Theory

The basin exists before the trajectories converge on it. The casque was buried before anyone read the painting. The ground state was there before the observational plane was occupied.

Submission date is movement. The paper is vibration. The truth — the ground state — precedes both.

eaiou evaluates the vibration, not when the movement happened.

SAID Witness

> “We make submission dates immutable state space only observable to the layer above and nobody else — that way you don’t read science by time.” — Eric D. Martin, 2026-04-06, ORCID: 0009-0006-5944-1742

0. these are preliminary brainstorms. YOu need to vcreae a SSOT from this si we ccan build the website jourmnal up from Joomla 5.3 under LAMP stack using the JOomla artivcle system. acl as teh foundation for th4 journal;, adding to the acl, and using the acl rather than repcle it and using 4h article systems as is and adding to tits core functions usiong custom plugins per feature so we can easily debug lets say research-endpoints)

1.) Treat things that can talk like humanity.

---

2. (not exhaustiove)EAIOU Feature Inventory (from SQL)

---

eaiou_papers

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)

- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Paper metadata (title, abstract, authors, doi)
- Authorship tracking (authorship_mode)
- Submission lifecycle fields (status, submission/acceptance/publication dates)
- Version tagging (submission_version)

eaiou_remsearch

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id (link to parent records)
- Literature triage (citation_title/source/link)
- Inclusion triage (used flag, reasons)
- Source typing (source_type)

eaiou_ai_sessions

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id (link to parent records)
- AI session labeling (session_label, ai_model_name)
- Session timing (start_time, end_time) and token counts
- Redaction workflow (redaction_status, session_notes, session_hash)

eaiou_didntmakeit

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)

- Relations: session_id (link to parent records)
- Excluded prompt/response archival
- Redaction tracking (redacted, redaction_hash)

eaiou_plugins_used

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id (link to parent records)
- Plugin/tool audit (plugin_name, plugin_type, execution_context)
- Execution logs per run

eaiou_review_logs

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id, reviewer_id (link to parent records)
- Peer review event log (review_date, reviewer_id, version_reviewed)
- Review rubric (overall, originality, methodology, transparency, AI disclosure)
- Narrative notes (review_notes)

eaiou_versions

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id (link to parent records)
- Version labeling & file path
- AI authorship flag + model name
- Generation date and notes

eaiou_attribution_log

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: paper_id (link to parent records)
- Attribution entries (contributor_name, role_description, contribution_type)
- AI/human authorship booleans
- Timestamped linkage to paper/version
- Tool reference (ai_tool_used)

eaiou_api_keys

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: user_id (link to parent records)
- API key registry (api_key, description)
- Key owner (user_id) and status
- Access tiering (access_level)

eaiou_api_logs

- CRUD: create/read/update/delete records for this entity
- Publishing state toggle (state)
- Access level control (access)
- Sortable ordering (ordering)
- Audit fields (created/modified/checked_out timestamps and users)
- Relations: api_key_id (link to parent records)
- API call audit (endpoint, request_data, response_code, timestamp)
- Key linkage (api_key_id)

Features of peer review websites

Peer review websites, primarily used in academic publishing, facilitate the submission, review, and publication process for scholarly work.

Key features include:

1. Author submission portal

Allows authors to upload manuscripts, supplementary files, and essential metadata.

Supports various formats for manuscripts, figures, and supplemental materials, including Microsoft Word, TeX, PDF, TIFF, EPS, BMP, PNG, JPG, and video, audio, and data sets.

Provides real-time status updates on submissions and streamlines the process through an intuitive interface.

2. Peer review management

Facilitates the selection and assignment of reviewers.

Tracks review progress and enables reviewers to submit feedback.

Supports different peer review models, including single-blind, double-blind, transparent, collaborative, and post-publication review.

Ensures confidentiality and protects against reviewer bias through features like anonymized review processes and conflict of interest disclosures.

Provides tools and guidelines to assist reviewers in delivering constructive and ethical feedback, according to Yomu AI.

3. Editorial workflow

Allows editors to make decisions, communicate with authors, and manage revisions.

Manages the communication between authors, reviewers, and editors through integrated email systems and automated notifications.

Offers dashboards and reports for tracking progress and analyzing metrics like response times and review quality.

4. Integrity and quality assurance

Integrates with plagiarism detection tools and formatting checks to enforce journal-specific guidelines.

Uses reviewer databases to select qualified reviewers and monitor their performance.

Offers features like co-author verification to prevent misconduct like ghost or gift authorship.

5. Other important features

Automated correspondence and reminders for deadlines and decisions.

Integration with research databases like ORCID and CrossRef for author identification and citation linking.

Open-source platforms (like Open Journal Systems) offer flexibility and customization for smaller journals and academic institutions.

In essence, these websites are designed to streamline the peer review process, ensuring efficiency, quality, and integrity in academic publishing.

AI responses may include mistakes. Learn more

What plagiarism detection tools are integrated into peer review websites?

Elaborate on features ensuring reviewer confidentiality and preventing bias

How do peer review websites handle manuscript revisions?

undefined

undefined

undefined

11 sites

How To Use Manuscript Submission Systems for Papers

Jul 18, 2024 — What Are Manuscript Submission Systems? Manuscript submission systems are online platforms designed to facilitate the submission, review, and publication of res…

favicon

www.journal-publishing.com

Editorial Management System for peer review: online?

Oct 20, 2017 — Features to look for * An online submission form for authors to submit manuscripts with metadata. * A dashboard for editorial staff to track submissions at vari…

favicon

Manuscript Manager

Types of Peer Review - Wiley Author Services

Types of peer review. The three most common types of peer review are single-anonymized, double-anonymized, and open peer review. Peer review is constantly evolv…

favicon

Wiley Author Services

Show all

i need an ecxhaustive feature lizst

Exhaustive list of peer review website features

Peer review websites are crucial for the academic publishing lifecycle, offering a comprehensive suite of features that span from author submission to publication and reporting. These features aim to enhance the efficiency, transparency, and integrity of the peer review process.

Here’s an exhaustive list of key features:

1. Author and reviewer management

Submission Portal: Intuitive interface for authors to submit manuscripts, including various file types (Word, PDF, LaTeX, figures, data sets, etc.).

Reviewer Databases: Maintain detailed profiles of potential reviewers, including their areas of expertise, review history, and performance metrics.

Conflict of Interest Checks: Features to identify and manage potential conflicts of interest among authors, reviewers, and editors.

Anonymization Options: Support for different peer review models, including single-blind, double-blind, and triple-blind, to ensure impartiality by masking identities, according to the University of California.

2. Editorial workflow and process management

Workflow Customization: Configurable workflows to adapt to specific journal requirements and processes.

Automated Assignment: Smart algorithms to match submissions with suitable reviewers based on expertise and workload.

Deadline Tracking and Reminders: Automated notifications and reminders for authors, reviewers, and editors to ensure timely progress.

Communication Tools: Built-in messaging systems and email templates to facilitate communication among all stakeholders.

Revision and Version Control: Tools to manage revisions, track changes, and maintain a history of manuscript versions.

Decision Management: Streamlined process for editors to make decisions (accept, reject, revise), communicate feedback, and manage revisions.

Editorial Board Management: Tools to manage editorial board members, their roles, and permissions.

3. Quality and integrity assurance

Plagiarism Detection: Integration with plagiarism detection services (like Crossref’s Similarity Check) to screen submissions, according to scholasticahq.com.

Format Checking: Tools to ensure submitted manuscripts adhere to journal formatting guidelines.

Ethical Guidelines Enforcement: Features that support and enforce ethical guidelines for peer review, including reviewer guidelines and misconduct reporting.

Transparency and Open Peer Review Options: Support for transparent or open peer review models where review reports and author responses may be published alongside the accepted manuscript.

4. Reporting and analytics

Performance Tracking: Dashboards and reports to monitor key metrics, including submission-to-decision times, reviewer performance, and manuscript status.

Data Export: Options to export data for further analysis or archiving.

5. Security and data privacy

Secure Data Handling: Encryption of data in transit and at rest, strong authentication measures (e.g., two-factor authentication), and access control mechanisms.

Role-Based Permissions: Assigning granular permissions to users based on their roles (authors, reviewers, editors) to control data access.

Compliance: Adherence to data protection regulations like GDPR and HIPAA.

Data Backup and Recovery: Mechanisms for regular data backups and disaster recovery planning.

6. Integration and customization

Persistent Identifier Integration: Integration with ORCID and Crossref to uniquely identify authors and link publications and reviews.

Customizable Forms and Templates: Ability to create custom submission forms, review forms, and email templates.

Branding and Customization: Options to customize the platform’s appearance and branding to align with journal identity.

7. Additional features

LMS Integration: Integration with learning management systems (LMS) for educational peer review scenarios.

Accessibility and Mobile Responsiveness: Ensuring the platform is accessible and usable across various devices and for individuals with disabilities.

This comprehensive list highlights the many features that contribute to a robust and effective peer review website.

AI responses may include mistakes. Learn more

Exhaustive list of peer review website features (with granular details)

1. Author and reviewer management

1.1. Submission portal

1.1.1. File upload capabilities: Support for diverse file formats (e.g., Word, PDF, LaTeX, figures in TIFF, EPS, PNG, JPG, multimedia files like video/audio, and data sets).

1.1.2. Metadata capture: Fields for capturing essential metadata such as author information (including ORCID iDs), affiliations, keywords, abstract, and article type.

1.1.3. Submission form customization: Ability to add customized affirmations, file upload requirements, instructional text, CRediT (Contributor Role Taxonomy) fields, and more according to Scholastica.

1.1.4. Real-time status updates: Automated notifications and a dedicated interface for authors to track the progress of their submissions (e.g., received, under review, decision made).

1.1.5. Version control: Allows authors to upload revised manuscripts and supplementary materials while maintaining a clear history of previous versions.

1.2. Reviewer databases

1.2.1. Detailed reviewer profiles: Store information on reviewer expertise (using keywords or tagging functionality), institutional affiliations, contact information, and review history.

1.2.2. Performance tracking: Record metrics such as the number of completed reviews, average review completion time, number of on-time reviews, and decline rate.

1.2.3. Availability and workload management: Track reviewer availability and current workload to avoid over-inviting or assigning manuscripts that conflict with their schedules.

1.2.4. Conflict of interest tracking: Flag potential conflicts based on co-authorships, affiliations, or other relevant information.

1.2.5. Incentives and recognition: Integration with services like Publons or ReviewerCredits to acknowledge and reward reviewers for their contributions.

1.3. Conflict of interest checks

1.3.1. Automated detection: Flag potential conflicts based on keywords in submitted manuscripts or reviewer profiles.

1.3.2. Manual review and disclosure: Prompt authors, reviewers, and editors to disclose any potential conflicts of interest during the submission and review process.

1.3.3. Mitigation strategies: Tools to manage identified conflicts (e.g., re-assigning reviewers, informing the editorial board).

1.4. Anonymization options

1.4.1. Single-blind review: Masking reviewer identities from authors.

1.4.2. Double-blind review: Masking both author and reviewer identities from each other.

1.4.3. Triple-blind review: Masking author, reviewer, and editor identities from each other (less common but possible).

1.4.4. De-identification tools: Automated tools to remove identifying information from manuscripts and supplementary files to support blind review processes.

2. Editorial workflow and process management

2.1. Workflow customization

2.1.1. Configurable stages and statuses: Customize the different stages of the review process (e.g., initial check, reviewer invitation, review in progress, decision) and their associated statuses.

2.1.2. Role-based permissions: Define granular permissions for different user roles (e.g., editors, associate editors, journal administrators) at each workflow stage.

2.1.3. Automated actions: Automate tasks based on specific triggers (e.g., sending reviewer reminders, moving manuscripts to the next stage upon completion of reviews).

2.1.4. Adaptable templates: Create and customize templates for various communications (e.g., invitation letters, decision letters).

2.2. Automated assignment

2.2.1. Smart matching algorithms: Algorithms that suggest reviewers based on their areas of expertise (keywords, publication history), review performance, and current workload.

2.2.2. Load balancing: Distribute review invitations equitably among qualified reviewers.

2.2.3. Reviewer preferences: Factor in reviewer preferences (e.g., topics of interest, preferred review load) when suggesting assignments.

2.3. Deadline tracking and reminders

2.3.1. Configurable deadlines: Set deadlines for each stage of the review process (e.g., reviewer invitation response, review submission, revision submission).

2.3.2. Automated reminders: Send automated email reminders to authors, reviewers, and editors as deadlines approach or are missed.

2.3.3. Customizable notification frequency: Allow users to adjust the frequency of email reminders based on their preferences.

2.4. Communication tools

2.4.1. Secure messaging system: Built-in platform for secure communication between authors, reviewers, and editors, according to scholasticahq.com.

2.4.2. Email templates and personalization: Easily create and manage email templates for various correspondences (e.g., invitations, decision letters) with options for personalization.

2.4.3. Commenting and annotation tools: Enable reviewers to provide feedback directly on the manuscript within the platform.

2.5. Revision and version control

2.5.1. Track changes functionality: Allow authors and reviewers to track changes made to the manuscript during the revision process.

2.5.2. Version history and comparison: Maintain a clear history of all manuscript versions and offer tools to compare different versions.

2.5.3. Author response management: Provide a structured way for authors to respond to reviewer comments and indicate how they have addressed the feedback.

2.6. Decision management

2.6.1. Pre-defined decision options: Offer a range of decision options (e.g., accept, accept with minor revisions, accept with major revisions, revise and resubmit, reject).

2.6.2. Customizable decision letters: Create personalized decision letters that incorporate reviewer comments and editorial feedback.

2.6.3. Decision approval workflows: Require approval from senior editors or the editorial board for certain decisions.

2.7. Editorial board management

2.7.1. User role assignment: Assign specific roles (e.g., editor-in-chief, associate editor, guest editor) with corresponding permissions.

2.7.2. Performance tracking: Monitor the performance of editorial board members (e.g., decision-making speed, reviewer selection).

3. Quality and integrity assurance

3.1. Plagiarism detection

3.1.1. Integration with plagiarism detection services: Seamlessly connect with services like iThenticate to screen submissions for plagiarism, notes scholasticahq.com.

3.1.2. Similarity reports: Provide detailed reports highlighting instances of potential plagiarism.

3.1.3. Automated checks: Automate plagiarism screenings for all new submissions.

3.2. Format checking

3.2.1. Pre-submission checks: Automated tools to verify that submitted manuscripts adhere to journal-specific formatting guidelines (e.g., citation style, figure resolution).

3.2.2. Error notifications: Notify authors of any formatting errors detected during the submission process.

3.3. Ethical guidelines enforcement

3.3.1. Reviewer guidelines and codes of conduct: Provide clear guidelines for ethical conduct during peer review.

3.3.2. Misconduct reporting mechanisms: Offer channels for reporting suspected misconduct by authors, reviewers, or editors.

3.3.3. Co-author verification: Tools to verify the contributions of each author and prevent issues like ghost or gift authorship.

3.4. Transparency and open peer review options

3.4.1. Open reports: Publish review reports alongside the final published article.

3.4.2. Open identities: Allow authors and reviewers to be aware of each other’s identities.

3.4.3. Post-publication commenting: Enable commenting and discussion on published articles, including the version of record.

4. Reporting and analytics

4.1. Performance tracking

4.1.1. Key metrics: Track metrics like submission-to-decision time, rejection rates, acceptance rates, and reviewer turnaround times.

4.1.2. Customizable dashboards: Provide interactive dashboards for visualizing key performance indicators.

4.1.3. Drill-down capabilities: Allow users to drill down into specific data points for more granular analysis.

4.2. Data export

4.2.1. Customizable reports: Generate reports in various formats (e.g., CSV, Excel) with customizable data fields.

4.2.2. API access: Provide API access for integrating with other systems and exporting data programmatically.

5. Security and data privacy

5.1. Secure data handling

5.1.1. Encryption: Encrypt data both in transit and at rest to protect sensitive information.

5.1.2. Access control: Implement granular, role-based access control mechanisms to restrict access to sensitive data and functionality.

5.1.3. Authentication: Support strong authentication methods, including multi-factor authentication.

5.2. Role-based permissions

5.2.1. Granular permission sets: Define fine-grained permissions for different user roles and workflow stages.

5.2.2. Customizable roles: Allow administrators to create custom roles and assign specific permissions based on organizational needs.

5.3. Compliance

5.3.1. Regulatory adherence: Ensure compliance with relevant data privacy regulations like GDPR and HIPAA.

5.3.2. Audit trails: Maintain detailed audit trails of user actions and system changes to support compliance and accountability.

5.4. Data backup and recovery

5.4.1. Automated backups: Implement automated backup procedures to regularly create copies of data and configuration settings.

5.4.2. Disaster recovery planning: Establish a disaster recovery plan to ensure business continuity in the event of data loss or system failure.

6. Integration and customization

6.1. Persistent identifier integration

6.1.1. ORCID integration: Connect with ORCID to uniquely identify authors and link their publications and reviews.

6.1.2. Crossref integration: Integrate with Crossref for automated DOI registration and citation linking.

6.2. Customizable forms and templates

6.2.1. Flexible form designers: Visually design and customize submission forms, review forms, and other data input forms.

6.2.2. Template libraries: Provide libraries of pre-designed templates for common use cases.

6.3. Branding and customization

6.3.1. Interface customization: Customize the platform’s appearance and branding to align with journal or publisher identities.

6.3.2. Custom domain support: Host the platform on a custom domain.

7. Additional features

7.1. LMS integration

7.1.1. Learning management system integration: Integrate with platforms like Canvas to facilitate educational peer review processes.

7.1.2. Gradebook synchronization: Synchronize grades and feedback from peer reviews with the LMS gradebook.

7.2. Accessibility and mobile responsiveness

7.2.1. Accessibility standards adherence: Ensure compliance with accessibility guidelines (e.g., WCAG) to make the platform usable for individuals with disabilities.

7.2.2. Responsive design: Optimize the user interface for seamless use across various devices (e.g., desktops, tablets, mobile phones).

1. Application: aieou.org

6.1 Concept

aieou.org is conceived as a Layer 5 domain module for the Meta Modal Platform (oMMP), serving as a publishing and archival platform that enforces full-context submission of research materials. In contrast to conventional journals that prioritize concise, polished narratives, aieou.org requires the inclusion of all relevant artifacts generated during the research process — whether ultimately used in the final analysis or not. This aligns with emerging reproducibility mandates in open science, which emphasize the value of sharing null results, exploratory models, and intermediate datasets (Munafò et al., 2017).OMMP_framework.thesis

6.2 Features

Mandatory Full-Context Submission

All research material, including failed trials, calibration data, and exploratory notes, must accompany the primary submission. This preserves un space as defined in oMMP, ensuring that no observer-linked variables are lost.

Searchable Index of un Research

A dedicated search engine indexes both “used” and “unused” research materials. Researchers can search by domain, methodology, or observer ID, allowing connections between seemingly unrelated projects.

Structured Reviewer Options

Reviewers can:

- Credit the work as relevant,
- Mark it as not relevant (without removal), or
- Provide structured critique for archival with the submission.

This process mirrors best practices in transparent peer review (Ross-Hellauer, 2017).

AI Usage Logging

All AI-assisted contributions are logged in a controlled research environment, ensuring reproducibility and compliance with emerging guidelines on AI in research (Nature Editorial, 2023).OMMP_framework.thesis

6.3 Role in Closing the un Space Gap

By enforcing comprehensive submission and preservation of the observer’s entire research path, aieou.org functions as an observer-preserving publishing layer. This directly addresses the innovation shadow zone described in Section 5.2, preventing the silent loss of epistemically valuable material.

Such a system would also enable cross-domain serendipity: a dataset deemed irrelevant in one field might hold the key to a breakthrough in another. In this way, aieou.org extends oMMP’s reach beyond experimental science into the broader academic ecosystem, transforming the un space from an inaccessible void into a searchable, interconnected knowledge layer.OMMP_framework.thesis

10.3 Development of aieou.org as a Formal Publishing Platform

Beyond its role as a Layer 5 oMMP module, aieou.org could evolve into a global open science repository with advanced un space search capabilities. By allowing research institutions, governments, and independent researchers to submit both used and unused data under a unified metadata schema, aieou.org could become a central node in the global research network. Integration with entropy trace mapping tools could make it the first publishing platform to quantify the completeness of preserved observational context.OMMP_framework.thesis

6. Application: aieou.org

6.1 Concept

aieou.org is conceived as a Layer 5 domain module for the Meta Modal Platform (oMMP), serving as a publishing and archival platform that enforces full-context submission of research materials. In contrast to conventional journals that prioritize concise, polished narratives, aieou.org requires the inclusion of all relevant artifacts generated during the research process — whether ultimately used in the final analysis or not. This aligns with emerging reproducibility mandates in open science, which emphasize the value of sharing null results, exploratory models, and intermediate datasets (Munafò et al., 2017).OMMP_framework.thesis

6.2 Features

Mandatory Full-Context Submission

All research material, including failed trials, calibration data, and exploratory notes, must accompany the primary submission. This preserves un space as defined in oMMP, ensuring that no observer-linked variables are lost.

Searchable Index of un Research

A dedicated search engine indexes both “used” and “unused” research materials. Researchers can search by domain, methodology, or observer ID, allowing connections between seemingly unrelated projects.

Structured Reviewer Options

Reviewers can:

- Credit the work as relevant,
- Mark it as not relevant (without removal), or
- Provide structured critique for archival with the submission.

This process mirrors best practices in transparent peer review (Ross-Hellauer, 2017).

AI Usage Logging

All AI-assisted contributions are logged in a controlled research environment, ensuring reproducibility and compliance with emerging guidelines on AI in research (Nature Editorial, 2023).OMMP_framework.thesis

6.3 Role in Closing the un Space Gap

By enforcing comprehensive submission and preservation of the observer’s entire research path, aieou.org functions as an observer-preserving publishing layer. This directly addresses the innovation shadow zone described in Section 5.2, preventing the silent loss of epistemically valuable material.

Such a system would also enable cross-domain serendipity: a dataset deemed irrelevant in one field might hold the key to a breakthrough in another. In this way, aieou.org extends oMMP’s reach beyond experimental science into the broader academic ecosystem, transforming the un space from an inaccessible void into a searchable, interconnected knowledge layer.OMMP_framework.thesis

10.3 Development of aieou.org as a Formal Publishing Platform

Beyond its role as a Layer 5 oMMP module, aieou.org could evolve into a global open science repository with advanced un space search capabilities. By allowing research institutions, governments, and independent researchers to submit both used and unused data under a unified metadata schema, aieou.org could become a central node in the global research network. Integration with entropy trace mapping tools could make it the first publishing platform to quantify the completeness of preserved observational context.OMMP_framework.thesis

Proposed Research Endpoints for aieou.org (Layer 5 OMMP)

/submit — Endpoint for authenticated observer submissions (supports full-context bundle: primary data, unused data, calibration logs, AI contributions).

/review — Structured reviewer interface allowing “credit,” “not relevant,” or “critique” actions; all actions archived without deletion.

/search — Query index for “used” and “unused” research; supports domain, method, and observer-ID filters.

/trace/entropy — Generates Entropy Trace Maps (ETMs) to visualize ESR metric compliance across a submission’s lifecycle.

/dataset/link — Cross-links datasets that share observer IDs, Meta tokens, or Modal codes.

/ai/log — Returns AI-assist logs for a given submission, ensuring compliance with AI usage transparency requirements.

/export/context — Exports preserved un space context for integration with other OMMP-compliant archives.

/register/observer — Registers cryptographic observer identities (Ed25519 PKI) for attribution and reproducibility.

If you want, I can also merge these endpoints into a full Layer 5 OMMP spec section so they’re embedded directly into the framework’s normative architecture. That way, aieou isn’t just a module note — it becomes a fully formalized part of OMMP’s operational layer. Would you like me to do that?

/submit — Endpoint for authenticated observer submissions (supports full-context bundle: primary data, unused data, calibration logs, AI contributions).

/review — Structured reviewer interface allowing “credit,” “not relevant,” or “critique” actions; all actions archived without deletion.

/search — Query index for “used” and “unused” research; supports domain, method, and observer-ID filters.

/trace/entropy — Generates Entropy Trace Maps (ETMs) to visualize ESR metric compliance across a submission’s lifecycle.

/dataset/link — Cross-links datasets that share observer IDs, Meta tokens, or Modal codes.

/ai/log — Returns AI-assist logs for a given submission, ensuring compliance with AI usage transparency requirements.

/export/context — Exports preserved un space context for integration with other OMMP-compliant archives.

/register/observer — Registers cryptographic observer identities (Ed25519 PKI) for attribution and reproducibility.

Added Research Discovery & Collaboration Endpoints

/research/open — Marks specific datasets, hypotheses, or methods as open for collaboration. Includes optional tags:

interest_level (high/medium/low)

collaboration_type (co-author, data-sharing, peer review, funding partner)

notes (free-text intent statement).

/research/seek — Allows an observer to declare a research need (equipment, domain expertise, analysis partner, funding, etc.).

/ideas/discover — Returns a list of emerging research ideas from unused datasets and exploratory notes in the archive, ranked by entropy-novelty metrics.

/ideas/subscribe — Push or pull feed of new high-entropy ideas in a user-selected domain, filtered by ESR scores.

/collaboration/match — Suggests possible collaborators based on:

Shared Meta tokens (physical context)

Overlapping Modal codes (phenomenon structure/function)

Observer skill tags and prior work history.

/trend/insight — Aggregates and surfaces trending underexplored topics across the archive, based on search queries and open research declarations.

/gap/map — Generates visual GitGap-style maps of un space regions where no research has been done, but related data exist in unused archives.

Plugin 01 — Transparency Block (plg_content_transparency)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, no custom DB tables
Plugin Type: `content`
Goal: Render and validate structured Source/Data/Method transparency on Articles using native Custom Fields + Workflows.

---

1) Purpose

- Enforce and render a **Transparency Block** on Article pages:
  - **Sources consulted** (used and excluded) with reasons.
  - **Datasets & code** (location, license, accessibility).
  - **Method & limitations** notes.
- Provide **pre-publish validation** to prevent workflow transition unless required transparency fields are complete.
- Zero new schema: use **Custom Fields** assigned to Articles + native Action Logs.

---

2) Core-first Dependencies

- **com_content (Articles)** for submissions.
- **Custom Fields** (repeatable groups).
- **Workflows** for state transitions.

- **Action Logs** for provenance.
- **Email Templates** (optional) to notify authors of missing fields.

---

3) Events & Hooks (Joomla 5 typed events)

- View/render injection:
  - `onContentAfterTitle` → add compact transparency badge/summary.
  - `onContentAfterDisplay` → render full Transparency Block.
- Validation/Enforcement:
  - `onContentPrepareForm` → inject field hints + client-side help.
  - `onContentBeforeSave` → server-side validation on save.
  - `onContentBeforeChangeState` → block state change if required fields missing.
- Data normalization:
  - `onContentNormaliseRequestData` → trim lists, normalize URLs/DOIs.

> **Note:** Use the new event classes (e.g., `\Joomla\CMS\Event\Model\BeforeSaveEvent`) where applicable.

---

4) Content Model (Custom Fields spec)

Create a **Field Group: Transparency** (Article context).

A. Sources (Repeatable)

- `transp_sources` (Repeatable Subform):
  - `src_title` (Text, required)
  - `src_type` (List: Primary, Secondary, Tertiary)
  - `src_pid` (Text, DOI/Handle/URL)
  - `src_used` (Radio: Used / Excluded)
  - `src_reason` (Textarea — required if Excluded)
  - `src_notes` (Textarea, optional)

B. Datasets (Repeatable)

- `transp_datasets` (Repeatable Subform):
  - `data_title` (Text, required)

    - `data_link` (URL)
    - `data_type` (List: Raw, Processed, Code, Images, Other)
    - `data_license` (Text/List — SPDX optional)
    - `data_availability` (List: Open, Restricted, Upon Request)
    - `data_anonymization` (Checkbox + Textarea notes)

C. Methods & Limitations

- `transp_methods` (Textarea, required)
- `transp_limitations` (Textarea, required)

D. Integrity Flags

- `transp_complete` (Checkbox — auto-set by plugin when valid)
- `transp_lastcheck` (Calendar — auto-updated on validation)

---

5) Validation Rules

- At least **one Source** entry.
- All **Excluded** sources must include `src_reason`.
- If **Datasets** present and `data_availability=Restricted/Upon Request` → require a short justification in notes.
- `transp_methods` and `transp_limitations` **required** before publishing (state change to Published).
- On pass: set `transp_complete=1`, update `transp_lastcheck`.
- On fail: raise message; block save or state transition (depending on context).

---

6) Rendering (Front-end)

- **Badge line (after title):** small icons + counts: Sources Used/Excluded, Datasets, Last Checked date, completeness.
- **Full block (after content):**
    - Collapsible panels: **Sources**, **Datasets**, **Methods & Limitations**.
    - Each source shows title → link (DOI/URL), type, status (Used/Excluded) + reason if excluded.
    - Respect article view access and ACL.

Use **WebAsset API** to enqueue minimal JS (accordion) and CSS.

---

7) ACL & Workflows

- Only **Editors+** can override validation failures (via a toggle in plugin params: “Allow editorial override”).
- Authors can save drafts with missing transparency; **publish** requires completeness.
- Integrates with **Workflows**: check on transitions `Draft → Under Review`, `Under Review → Published`.

---

8) Action Logs

- On successful validation/publish, log a summary line:
  - `TRANSPARENCY_OK: sources={used:X, excluded:Y}, datasets=Z, lastcheck=ISO8601`
- On block, log `TRANSPARENCY_FAIL` with reasons (not field contents).

---

9) Email Templates (optional)

- `tpl_transparency_missing`: sent to Author when editor attempts to move forward but fields missing.

---

10) Internationalization (i18n)

Language keys (example):

```
PLG_CONTENT_TRANSPARENCY="Transparency Block"
PLG_CONTENT_TRANSPARENCY_BADGE="Transparency"
PLG_CONTENT_TRANSPARENCY_SOURCES="Sources"
PLG_CONTENT_TRANSPARENCY_EXCLUDED="Excluded"
PLG_CONTENT_TRANSPARENCY_USED="Used"
PLG_CONTENT_TRANSPARENCY_DATASETS="Datasets & Code"
PLG_CONTENT_TRANSPARENCY_METHODS="Methods"
PLG_CONTENT_TRANSPARENCY_LIMITATIONS="Limitations"
PLG_CONTENT_TRANSPARENCY_INCOMPLETE="Transparency requirements are incomplete."
```

Provide `en-GB.plg_content_transparency.ini` and `.sys.ini`.

---

11) File/Folder Scaffold

```
/plugins/content/transparency/transparency.php
/plugins/content/transparency/transparency.xml
/plugins/content/transparency/services/provider.php
/plugins/content/transparency/language/en-GB/en-GB.plg_content_transparency.ini
/plugins/content/transparency/language/en-GB/en-
GB.plg_content_transparency.sys.ini
/plugins/content/transparency/tmpl/block.php
/plugins/content/transparency/media/css/transparency.css
/plugins/content/transparency/media/js/transparency.js
```

- `provider.php` registers services & event listeners (J5 style).
- `tmpl/block.php` renders the full block using Article’s Custom Fields.

---

12) Minimal Manifest (transparency.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="content" method="upgrade" version="5.0">
  <name>plg_content_transparency</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Renders and validates transparency metadata for Articles.</
        description>
  <files>
    <filename plugin="transparency">transparency.php</filename>
    <filename>services/provider.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_transparency.ini</
        language>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_transparency.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="show_badge" type="radio" default="1" label="Show Badge"
        description="Show compact badge after title."/>
        <field name="editor_override" type="radio" default="0" label="Allow
        Editorial Override" description="Permit Editors to publish even if
        incomplete."/>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

13) Test Plan

Unit/Integration - Mock events: `onContentBeforeSave` rejects when required fields missing. - Accepts when required fields filled; sets `transp_complete` and `transp_lastcheck`.

Functional - Author saves draft missing fields (allowed). - Editor attempts to publish → blocked; email sent (optional). - After completing fields, publish succeeds; Action Log populated. - Frontend renders badge + block; toggles behave.

Regression - Ensure rendering only on Article views (not categories/feeds unless enabled). - Confirm compatibility with template overrides.

---

14) Rollout & Admin Checklist

1. Create Field Group “Transparency” and fields from §4.
2. Enable plugin; set params.
3. Update Workflows to include validation on transitions.
4. Add language overrides for front-end labels if needed.
5. Add menu item note: authors must complete Transparency before publish.

Plugin 02 — AI Usage Log (plg_content_aiusage)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, no custom DB tables
Plugin Type: `content`
Goal: Capture, validate, and render a first-class **AI Usage Log** for Articles using native Custom Fields, providing auditability and clear disclosure. No external storage: all data lives in Article Custom Fields + Action Logs.

---

1) Purpose

- Provide a structured, auditable record when **AI assisted** the research or drafting process.
- Enforce minimal disclosure before publish; render an **AI Relationship Statement** on the article.
- Support privacy by design (hash prompts, redact sensitive text, configurable display).

---

2) Core-first Dependencies

- **com_content (Articles)** as the single content store.

- **Custom Fields** (Article context) for AI metadata.
- **Workflows** for state transitions (enforcement).
- **Action Logs** for provenance.
- **Email Templates** (optional) for missing-info nudges.

---

3) Events & Hooks (Joomla 5 typed)

- Render:
  - `onContentAfterTitle` → compact "AI-Logged" badge + summary chips.
  - `onContentAfterDisplay` → full AI Usage panel.
- Validate / Enforce:
  - `onContentPrepareForm` → inject field hints / help text.
  - `onContentBeforeSave` → validate user input; derive hashes.
  - `onContentAfterSave` → write Action Log summary.
  - `onContentBeforeChangeState` → block publish if required AI fields missing when `ai_used=1`.
- Normalization:
  - `onContentNormaliseRequestData` → normalize URLs, map model names, trim lists.

> Use typed event classes (`\Joomla\CMS\Event\Model\BeforeSaveEvent`, etc.) where applicable in J5.

---

4) Content Model (Custom Fields spec)

Create **Field Group: AI Usage** (Article context).

A. Usage Switch

- `ai_used` (Radio: No/Yes) — **drives validation**.

B. Tooling (Repeatable: each AI tool/model used)

- `ai_tools` (Repeatable Subform):
  - `ai_vendor` (Text/List — e.g., OpenAI, Anthropic, Google, Local) **required**
  - `ai_model` (Text — e.g., GPT-4.1, Claude-3.5) **required**
  - `ai_version` (Text, optional)

    - `ai_mode` (List: Chat, API, Batch, Image, Audio, Code, Other)
    - `ai_endpoint` (URL/Text — optional, for API endpoint or app URL)
    - `ai_params` (Textarea — temperature, top_p, system prompt notes)

C. Prompts & Outputs (Repeatable, privacy-aware)

- `ai_interactions` (Repeatable Subform):
    - `ai_prompt_summary` (Textarea — high-level description, **no raw prompt**) **required**
    - `ai_prompt_hash` (Text — **SHA-256** of prompt text kept locally by author, not stored in raw form) **required if author used raw**
    - `ai_output_type` (List: Text, Outline, Code, Table, Figure, Data, Other) **required**
    - `ai_contribution_scope` (List: Ideation, Drafting, Editing, Translation, Data Processing, Analysis, Visualization, Coding, Other) **required**
    - `ai_human_oversight` (Textarea — how the author verified/edited the output) **required**
    - `ai_external_data_used` (Radio: No/Yes) — if Yes, requires `ai_data_sources`.
    - `ai_data_sources` (Textarea — brief description/links to non-article data passed to AI) *conditional*
    - `ai_risk_flags` (Checkboxes: Hallucination Risk, Sensitive Data Risk, Copyright Risk, Bias Risk, Other)
    - `ai_redactions` (Textarea — what was redacted and why)

D. Statement & Controls

- `ai_relationship_statement` (Textarea — one-paragraph disclosure) **required if ai_used=Yes**
- `ai_display_level` (List: Full, Summary Only, Hidden to Public) — **render controls** (admins/reviewers always see full).

E. Integrity Fields (auto-managed)

- `ai_log_complete` (Checkbox — set by plugin if all required fields are valid)
- `ai_log_lastcheck` (Calendar — auto-updated on validation)

---

5) Validation Rules

When `ai_used=Yes`: - At least one **Tool** and one **Interaction** entry.
- `ai_prompt_summary`, `ai_output_type`, `ai_contribution_scope`, `ai_human_oversight` **required** per interaction.
- If `ai_external_data_used=Yes` → `ai_data_sources` required.
- `ai_relationship_statement` required.
- Compute **SHA-256** for `ai_prompt_hash` if author provides raw prompt (hash locally in plugin; do **not** store raw).
- On pass: set `ai_log_complete=1` and update `ai_log_lastcheck`.
- On fail: raise message and block **publish** transition.

When `ai_used=No`: no enforcement; rendering suppressed.

---

6) Rendering (Front-end)

- **Badge (after title):** `AI-Logged` chip with hover popover: model(s), interactions count, last checked date.
- **Full Panel (after content):**
  - Tooling summary table (Vendor, Model, Mode).
  - Interactions list (Prompt Summary, Output Type, Contribution Scope, Oversight, Data Sources, Risk Flags).
  - AI Relationship Statement (blockquote).
  - Respect `ai_display_level`:
    - **Full:** render all.
    - **Summary Only:** render tools + interaction counts + statement (no details).
    - **Hidden to Public:** show badge to public; full panel only for Reviewer/Editorial ACL.

Use **WebAsset API** for minimal CSS/JS (accordion/popover).

---

7) ACL & Workflows

- Authors can set/edit fields; **Editors** can enforce and override (configurable).

- Workflows check on transitions: `Draft → Under Review`, `Under Review → Published`.
- Admins/Reviewers always see full details regardless of `ai_display_level`.

---

8) Action Logs

- On publish or validation pass, log: `AI_LOG_OK: tools={n}, interactions={m}, lastcheck=ISO8601, display={level}`
- On failure: `AI_LOG_FAIL` with reasons (no raw content).

---

9) Privacy & Security

- **No raw prompts** stored by default; only a **hash** and **summary**.
- Redaction support via `ai_redactions`.
- Option in plugin params: “Allow raw prompt capture (admin-only)” (default **off**).
- Clearly label any field that becomes public; all else defaults to internal if `Hidden to Public`.

---

10) Internationalization (i18n)

Example keys:

```
PLG_CONTENT_AIUSAGE="AI Usage Log"
PLG_CONTENT_AIUSAGE_BADGE="AI-Logged"
PLG_CONTENT_AIUSAGE_PANEL="AI Relationship & Usage"
PLG_CONTENT_AIUSAGE_TOOLS="AI Tools"
PLG_CONTENT_AIUSAGE_INTERACTIONS="AI Interactions"
PLG_CONTENT_AIUSAGE_OVERSIGHT="Human Oversight"
PLG_CONTENT_AIUSAGE_PROMPT_SUMMARY="Prompt Summary"
PLG_CONTENT_AIUSAGE_RISK_FLAGS="Risk Flags"
PLG_CONTENT_AIUSAGE_MISSING="AI usage details are incomplete."
```

---

11) File/Folder Scaffold

```
/plugins/content/aiusage/aiusage.php
/plugins/content/aiusage/aiusage.xml
/plugins/content/aiusage/services/provider.php
/plugins/content/aiusage/tmpl/panel.php
/plugins/content/aiusage/media/css/aiusage.css
/plugins/content/aiusage/media/js/aiusage.js
/plugins/content/aiusage/language/en-GB/en-GB.plg_content_aiusage.ini
/plugins/content/aiusage/language/en-GB/en-GB.plg_content_aiusage.sys.ini
```

- `provider.php` registers services & listeners (J5).
- `tmpl/panel.php` renders the full panel from Custom Fields.

---

12) Minimal Manifest (aiusage.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="content" method="upgrade" version="5.0">
  <name>plg_content_aiusage</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Captures, validates, and renders AI usage disclosure for
        Articles.</description>
  <files>
    <filename plugin="aiusage">aiusage.php</filename>
    <filename>services/provider.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_aiusage.ini</language>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_aiusage.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="show_badge" type="radio" default="1" label="Show Badge" />
        <field name="enforce_publish" type="radio" default="1" label="Enforce
        before publish" />
        <field name="editor_override" type="radio" default="0" label="Allow
        editorial override" />
        <field name="allow_raw_prompts" type="radio" default="0"
        label="Allow raw prompt capture (admin-only)" />
        <field name="default_display_level" type="list" default="summary"
        label="Default display level">
          <option value="full">Full</option>
          <option value="summary">Summary Only</option>
          <option value="hidden">Hidden to Public</option>
        </field>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

13) Test Plan

Unit/Integration

- `onContentBeforeSave` rejects when `ai_used=Yes` and required fields missing.
- Accepts when complete; sets `ai_log_complete` and `ai_log_lastcheck`.

- Ensures hash is computed when author provided a raw prompt string (if allowed).

Functional
- Author toggles `ai_used=Yes`, fills minimal fields → publish passes.
- `Display Level` honored for public vs Reviewer/Editorial views.
- Action Log entries appear with correct summary.

Regression
- Panel appears only on Article views; badge toggles with param; no DB schema created.

---

14) Rollout & Admin Checklist

1. Create Field Group “AI Usage” & fields in §4.
2. Enable plugin; set params (display level, enforcement).
3. Add workflow guard on transitions (Draft → Review, Review → Published).
4. Add a short **Author Guideline** block on disclosure practices.
5. (Optional) Create an Email Template for missing info reminders.

Plugin 03 — Un Scientific Flag (plg_content_unscientific)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, no custom DB tables
Plugin Type: `content` (with minimal `system` behaviors if needed)
Goal: Provide a first-class **“Un Scientific”** marker for Articles to transparently flag **contested, ambiguous, or accuracy-pending** sections—especially in **human–AI collaboration**—without deleting content. Preserve discoverability and auditability using **Custom Fields, Tags, Workflows, and Action Logs** only.

---

1) Purpose

- Mark content that resides in **oMMP un space**: epistemically non-null but unresolved/contested.

- Support **human–AI collaboration** contention (accuracy/attribution/clarification).
- Make flagged areas **visible to readers** (badge/panel), **actionable for editors** (required clarifications), and **searchable** (Smart Search + Tag).
- Never remove content; instead, record contention, reasoning, and resolution path.

---

2) Core-first Dependencies

- **com_content (Articles)** as the single store.
- **Custom Fields** (Article) for flag state, reasons, scopes, resolution.
- **Tags** (optional) to auto-apply `un-scientific`.
- **Workflows** to gate state transitions if unresolved.
- **Action Logs** for provenance of add/remove/resolve actions.
- **Email Templates** (optional) to notify authors/reviewers.

---

3) Events & Hooks (Joomla 5 typed)

- Render:
  - `onContentAfterTitle` → show **Un Scientific** badge when active.
  - `onContentAfterDisplay` → render **Flag Panel** (reasons, scopes, requested clarifications, status).
- Validate / Enforce:
  - `onContentPrepareForm` → inject helper hints; restrict who may set/clear flag by ACL.
  - `onContentBeforeSave` → normalize & validate fields; may set tag automatically.
  - `onContentBeforeChangeState` → optionally block **Publish** until `resolved=Yes` or `editor_override=1`.
- Normalize:
  - `onContentNormaliseRequestData` → canonicalize scopes, unique IDs for flagged sections (if using anchors).

---

4) Content Model (Custom Fields spec)

Create **Field Group: Un Scientific** (Article context).

A. Flag Switch

- `unsci_active` (Radio: No/Yes) — controls rendering/enforcement.

B. Scope & Reasons (Repeatable: each contention entry)

- `unsci_entries` (Repeatable Subform):
  - `unsci_scope` (List: Whole Article, Abstract, Methods, Results, Discussion, Figure, Table, Dataset, AI Contribution, Other) **required**
  - `unsci_anchor` (Text — optional fragment/ID to point to a specific section/element)
  - `unsci_reason` (Checkboxes: Accuracy Clarification, Missing Citation, Method Ambiguity, Data Quality, COI Concern, AI Attribution, Ethics, Other) **required**
  - `unsci_notes` (Textarea — brief explanation) **required**
  - `unsci_requested_action` (Checkboxes: Provide Citation, Add Method Detail, Supply Data Link, Run Replication, Reword Claim, Add AI Oversight Notes, Other)
  - `unsci_risk_level` (List: Low, Medium, High)
  - `unsci_createdby` (User — auto-set when created via plugin UI)
  - `unsci_createdat` (Calendar — auto-set)

C. Resolution

- `unsci_resolved` (Radio: No/Yes)
- `unsci_resolution_notes` (Textarea — what changed/why) **required if resolved**
- `unsci_resolvedby` (User — auto-set when resolved)
- `unsci_resolvedat` (Calendar — auto-set)

D. AI Collaboration Link (optional cross-ref)

- `unsci_related_ai` (Checkbox — “Flag relates to AI Usage”)
  - When checked, plugin renders quick link/context from **AI Usage** fields.

E. Display Controls

- `unsci_display_level` (List: Full, Summary Only, Hidden to Public) — reviewers/editors always see full.

F. Integrity (auto)

- `unsci_lastcheck` (Calendar — updated on each save)
- `unsci_complete` (Checkbox — indicates minimally valid record)

---

5) Validation Rules

When `unsci_active=Yes`: - At least **one** `unsci_entries` item with `scope` and `reason`.
- If `unsci_resolved=Yes` → `unsci_resolution_notes` required; set resolver and timestamp.
- On pass: set `unsci_complete=1` and update `unsci_lastcheck`.
- On fail: block **Publish** transition (configurable).

Optional behaviors (params): - **Auto-tag** Article with `un-scientific` when active; remove when fully resolved.
- **Auto-notify** authors when flagged or when resolution requested.

---

6) Rendering (Front-end)

- **Badge (after title)**: `Un Scientific` with color based on `risk_level` max; hover shows count of entries & status (Resolved/Unresolved).
- **Panel (after content)**:
  - Table/list of entries with: Scope, Reasons, Notes, Requested Action, CreatedBy/At.
  - If `unsci_resolved=Yes`: show Resolution notes & resolver/time.
  - If `unsci_related_ai=Yes`: add a “See AI Usage” quick link.
  - Respect `unsci_display_level` (public vs reviewer/editor).

Use **WebAsset API** for minimal CSS/JS (accordion, status chips).

---

7) ACL & Workflows

- **Who can flag:** Reviewers, Editors, Admins (configurable); Authors may **view & respond** but cannot clear without Editor action (configurable).
- **Workflow gating:** if active & unresolved, block `Under Review → Published` unless `editor_override=1`.
- **Always visible** to Reviewer/Editorial roles regardless of display level.

---

8) Action Logs

- On flag set: `UNSCI_SET: entries={n}, maxrisk={level}`
- On resolve: `UNSCI_RESOLVED: entries={n}, notes_present={bool}`
- On publish attempt blocked: `UNSCI_BLOCKED: reason=unresolved`

No sensitive notes stored in logs; details remain in Custom Fields.

---

9) Search & Discoverability

- When active, **auto-apply Tag** `un-scientific` (config) → Smart Search filterable.
- Panel renders meta attributes for schema.org `CorrectionComment` (optional enhancement).

---

10) Internationalization (i18n)

Example keys:

```
PLG_CONTENT_UNSCIENTIFIC="Un Scientific Flag"
PLG_CONTENT_UNSCIENTIFIC_BADGE="Un Scientific"
PLG_CONTENT_UNSCIENTIFIC_PANEL="Contention & Clarifications"
PLG_CONTENT_UNSCIENTIFIC_SCOPE="Scope"
PLG_CONTENT_UNSCIENTIFIC_REASON="Reason"
PLG_CONTENT_UNSCIENTIFIC_REQUESTED="Requested Action"
PLG_CONTENT_UNSCIENTIFIC_RESOLUTION="Resolution"
PLG_CONTENT_UNSCIENTIFIC_BLOCK="Publishing blocked: outstanding Un Scientific
flags."
```

---

11) File/Folder Scaffold

```
/plugins/content/unscientific/unscientific.php
/plugins/content/unscientific/unscientific.xml
/plugins/content/unscientific/services/provider.php
/plugins/content/unscientific/tmpl/panel.php
/plugins/content/unscientific/media/css/unscientific.css
/plugins/content/unscientific/media/js/unscientific.js
/plugins/content/unscientific/language/en-GB/en-GB.plg_content_unscientific.ini
/plugins/content/unscientific/language/en-GB/en-
GB.plg_content_unscientific.sys.ini
```

---

12) Minimal Manifest (unscientific.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="content" method="upgrade" version="5.0">
  <name>plg_content_unscientific</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Flags contested or accuracy-pending content with Un Scientific
        markers.</description>
  <files>
    <filename plugin="unscientific">unscientific.php</filename>
    <filename>services/provider.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_unscientific.ini</
        language>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_unscientific.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="block_publish" type="radio" default="1"
        label="Block publish if unresolved" />
        <field name="editor_override" type="radio" default="0" label="Allow
        editorial override" />
        <field name="auto_tag" type="radio" default="1" label="Auto-apply 'un-
        scientific' tag" />
        <field name="default_display_level" type="list" default="summary"
        label="Default display level">
          <option value="full">Full</option>
          <option value="summary">Summary Only</option>
          <option value="hidden">Hidden to Public</option>
        </field>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

13) Test Plan

Unit/Integration - Saves with `unsci_active=Yes` and valid entry → OK; sets `unsci_complete` and timestamps.
- Publish blocked when unresolved and `block_publish=1`.
- Resolve flips state and clears tag if `auto_tag=1`.

Functional - Reviewer sets a flag with scope+reason → badge/panel show; author sees requested actions.
- Editor resolves with notes → publish allowed; Action Logs show events.
- `Display Level` honored for public vs reviewer/editor.

Regression - Rendering limited to Article views; no schema added; tag add/remove idempotent.

---

14) Rollout & Admin Checklist

1. Create Field Group "Un Scientific" & fields in §4.
2. (Optional) Create Tag `un-scientific`.
3. Enable plugin; set params (block publish, auto-tag).
4. Update Workflows (Review → Publish) to rely on plugin check.
5. Add short editorial SOP: when to flag, how to resolve, messaging templates.

Plugin 04 — Deadline Nudger (plg_system_deadline_nudger)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, **no custom DB tables**
Plugin Type: `system` (uses Joomla Scheduler)
Goal: Automate deadline reminders and escalations for submissions, reviews, and revisions using **native Articles, Custom Fields, Workflows, Mail Templates, and Scheduler**. All state lives in core; the plugin only reads fields, sends emails, and writes Action Logs.

---

1) Purpose

- Send **timely, templated reminders** for:
    1. **Reviewer invitation response due**
    2. **Review due**
    3. **Author revision due**
    4. Optional **editorial follow-ups** (stalled decisions)
- **Escalate** overdue items to editors after grace windows.

- Provide **dry-run** mode (logs only) for safe testing.
- Zero schema changes; **no new tables**.

---

2) Core-first Dependencies

- **com_content (Articles)** hold the submission record.
- **Custom Fields** (Article) store due dates, assignees, and status markers.
- **Workflows** define states (Draft → Review → Revisions → Accepted → Published).
- **Users/Groups/ACL** identify Authors, Reviewers, Editors, EIC.
- **Scheduler** (core) executes tasks; **Execution History** provides traceability.
- **Mail Templates** produce branded, localized emails.
- **Action Logs** record each notification or block.

---

3) Events & Hooks

- **Scheduler task** (primary): runs by CRON/webcron on the site, at configured intervals.
- **onAfterInitialise**: register scheduler task via services provider (J5).
- **onAfterRoute** (optional dev): expose a protected test endpoint for admins (toggle by plugin param).

---

4) Content Model (Custom Fields spec)

Create **Field Group: Deadlines** (Article context).

A. Reviewer Invitation

- `rev_invite_assignments` (Repeatable Subform):
  - `rev_user` (User) **required**
  - `rev_invite_sent` (Calendar)
  - `rev_invite_due` (Calendar) **required** (respond accept/decline)
  - `rev_invite_status` (List: Pending, Accepted, Declined, Replaced)
  - `rev_invite_escalated` (Checkbox auto)

B. Review Due

- `rev_due_date` (Calendar) — per article overall **or** per reviewer if needed:
  - If per reviewer is needed, add `rev_due_date` and `rev_due_escalated` inside **rev_invite_assignments**.

C. Author Revision

- `auth_rev_due` (Calendar)
- `auth_rev_escalated` (Checkbox auto)

D. Editorial Follow-up

- `ed_followup_due` (Calendar) — (e.g., **decision deadline** after last review)
- `ed_followup_escalated` (Checkbox auto)

> Note: Where the journal needs per-reviewer due dates, the plugin supports both **article-level** dates and **per-assignment** dates inside the repeatable field. Keep it simple at first; enable per-assignment via a param.

---

5) Scheduler Task Logic

Task ID: `plg_system_deadline_nudger.checkDue`

1. **Collect candidates** via a filtered Article query (core model) + Smart Search index (optional) using:
   - Category(ies) = journals/sections in scope (plugin param).
   - Workflow states: *Under Review*, *Revisions*, *Awaiting Decision* (param).
   - Presence of relevant **deadline fields**.
2. **Evaluate deadlines** for each candidate:
   - Compute **D-X**, **D-day**, **D+grace** windows (values from params).
   - Skip if already escalated/acknowledged (checkbox flags).
3. **Dispatch emails** using **Mail Templates** and **per-role recipients**:
   - Reviewer: invite response reminder / review due reminders.

    - Author: revision reminders.
    - Editor: escalations after grace.
    - Use language overrides and tokens (Article title, URL, due date, days late, reviewer name).

4. **Write Action Logs** for each message:

    - `NUDGER_SENT: context={invite|review|revision|editor}, user={id}, article={id}, due=ISO8601, deltaDays=N`
    - If dry-run: `NUDGER_DRYRUN: ...`

5. **Mark escalation** flags on success (set `_escalated` checkboxes) to avoid duplicate escalations.

6. **Summary**: push a one-line Scheduler history note with counts.

---

6) Email Templates (core Mail Templates)

Create these keys and templates in the admin (translatable, brandable): - `mail_reviewer_invite_reminder` — "Please accept/decline your review invitation"
- `mail_reviewer_due_reminder` — "Your review is due on {DATE}"
- `mail_author_revision_reminder` — "Your revision is due on {DATE}"
- `mail_editor_escalation` — "Escalation: {ARTICLE} is {N} days overdue for {CONTEXT}`

Tokens available (example): `{ARTICLE_TITLE}`, `{ARTICLE_LINK}`, `{DUE_DATE}`, `{DAYS_LATE}`, `{REVIEWER_NAME}`, `{AUTHOR_NAME}`, `{EDITOR_NAME}`

---

7) Params (admin)

- **Scope**: Category include/exclude; Workflow states include.
- **Windows**: Days before due to remind (e.g., 7, 3, 1), grace after due (e.g., 3, 7).
- **Per-reviewer mode**: On/Off.
- **Throttle**: max emails per run; min minutes between repeats to same user/article.
- **Dry-run**: On/Off (log only).

- **Test endpoint**: On/Off + Access Level.
- **Timezone**: force or inherit from site.
- **Copy to**: CC/BCC addresses (journal inbox).

---

8) ACL

- Only **Editors/EIC/Admin** can configure or run test endpoint.
- Emails respect **privacy**: recipients get only their relevant info (no PII leak).

---

9) UI/UX (no DB)

- **No admin component**; configuration is the plugin params UI.
- Optional small **dashboard module** in a later plugin for KPIs (not in scope here).

---

10) Internationalization (i18n)

Example keys:

```
PLG_SYSTEM_DEADLINE_NUDGER="Deadline Nudger"
PLG_SYSTEM_DEADLINE_NUDGER_DRYRUN="Dry run: no emails sent."
PLG_SYSTEM_DEADLINE_NUDGER_SUMMARY="Nudger run: {count} notifications, {esc}
escalations."
PLG_SYSTEM_DEADLINE_NUDGER_INVITE="Reviewer invite reminder"
PLG_SYSTEM_DEADLINE_NUDGER_REVIEW="Review due reminder"
PLG_SYSTEM_DEADLINE_NUDGER_REVISION="Author revision reminder"
PLG_SYSTEM_DEADLINE_NUDGER_ESCALATION="Editorial escalation"
```

Mail Templates are separate in the core Mail manager.

---

11) File/Folder Scaffold

```
/plugins/system/deadline_nudger/deadline_nudger.php
/plugins/system/deadline_nudger/deadline_nudger.xml
/plugins/system/deadline_nudger/services/provider.php
/plugins/system/deadline_nudger/language/en-GB/en-
GB.plg_system_deadline_nudger.ini
/plugins/system/deadline_nudger/language/en-GB/en-
GB.plg_system_deadline_nudger.sys.ini
```

- `provider.php` registers the **Scheduler task** and any event subscribers.

---

12) Minimal Manifest (deadline_nudger.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="system" method="upgrade" version="5.0">
  <name>plg_system_deadline_nudger</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Scheduler-based reminders and escalations for review workflows.</
        description>
  <files>
    <filename plugin="deadline_nudger">deadline_nudger.php</filename>
    <filename>services/provider.php</filename>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_system_deadline_nudger.ini</
        language>
    <language tag="en-GB">language/en-GB/en-
        GB.plg_system_deadline_nudger.sys.ini</language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="categories" type="category" multiple="true"
        extension="com_content" label="Categories in scope"/>
        <field name="states" type="text" default="under-
        review,revisions,awaiting-decision" label="Workflow states in scope"/>
        <field name="windows" type="text" default="7,3,1" label="Reminder days
        before due (CSV)"/>
        <field name="grace" type="number" default="3"
        label="Grace days after due for escalation"/>
        <field name="per_reviewer" type="radio" default="0" label="Per-reviewer
        due dates"/>
        <field name="throttle_max" type="number" default="100" label="Max emails
        per run"/>
        <field name="throttle_repeat_minutes" type="number" default="1440"
        label="Min minutes between repeats to same user/article"/>
        <field name="dry_run" type="radio" default="1" label="Dry run (log
        only)"/>
        <field name="test_endpoint" type="radio" default="0" label="Enable admin
        test endpoint"/>
        <field name="timezone" type="text" label="Timezone (leave blank to use
        site)"/>
        <field name="cc" type="text" label="CC addresses (CSV)"/>
        <field name="bcc" type="text" label="BCC addresses (CSV)"/>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

13) Test Plan

Unit/Integration
- Task finds due items and honors `dry_run`.
- Throttle prevents repeats; escalation flags set once.
- Per-reviewer mode correctly targets the right users.

Functional
- Reviewer receives invite and due reminders at D-7/3/1; grace triggers editor escalation.
- Author gets revision reminders; overdue triggers escalation.
- Scheduler history shows counts; Action Logs reflect sends.

Regression
- No emails to users without access to the article.
- No double-escalations; tags/flags idempotent; no schema changes.

---

14) Rollout & Admin Checklist

1. Create **Deadlines** Field Group & fields (§4).
2. Configure plugin params (categories, windows, grace, throttle).
3. Create **Mail Templates** (§6).
4. Enable the plugin; enable Scheduler CRON.
5. Run in **Dry-run** for 1–2 cycles; check logs; then turn on sending.

Plugin 06 — Open Reports (Transparent Peer Review Renderer) — `plg_content_openreports`

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, **no custom DB tables**
Plugin Type: `content`
Goal: Publish **peer-review reports** and author responses alongside the article using only **Articles, Custom Fields, Workflows, Tags, Action Logs, and Mail Templates**. Supports open/transparent, single-/double-blind, and “summary only” modes with privacy-safe redaction.

---

1) Purpose

- Enable **transparent peer review** by rendering reviewer reports and author responses on the public article view (when configured).
- Respect journal policies: **open identities**, **open reports**, or **summary only**.
- Enforce reviewer/author consent flow and **privacy-first redactions**.
- All data lives in **Article Custom Fields**; no new schema.

---

2) Core-first Dependencies

- **com_content (Articles)** as the canonical record.
- **Custom Fields** (Article context) for review entries and author responses.
- **Workflows** to guard publication of reports with consent checks.
- **Tags** (optional) to indicate `open-reports` capability per journal/section.
- **Action Logs** for provenance (publish, retract, redact).
- **Mail Templates** (optional) for consent requests and notifications.

---

3) Events & Hooks (Joomla 5 typed)

- Render:
  - `onContentAfterDisplay` → render the **Open Reports Panel** below the article body.
- Validate / Enforce:
  - `onContentPrepareForm` → inject field hints and surface policy notes.
  - `onContentBeforeSave` → validate reviewer consent flags and redaction settings.
  - `onContentBeforeChangeState` → block publish of reports if required consents are missing (configurable).
- Normalization:
  - `onContentNormaliseRequestData` → sanitize/strip email addresses/PII from display fields (configurable).

---

4) Content Model (Custom Fields spec)

Create **Field Group: Peer Review (Open Reports)** (Article context).

A. Policy Switches (per article)

- `or_enabled` (Radio: No/Yes) — enables output.
- `or_mode` (List: **Open Identities**, **Open Reports (anonymized)**, **Summary Only**) — controls visibility.
- `or_publish_on` (Radio: On Accept / On Publish / Manual) — when reports become public.
- `or_redact_emails` (Radio: Yes/No, default Yes) — automatic redaction.

B. Reviewer Reports (Repeatable)

- `or_reviews` (Repeatable Subform):
  - `or_rev_id` (Hidden; auto GUID)
  - `or_rev_user` (User — internal reference only; **never displayed** when anonymized)
  - `or_rev_displayname` (Text — shown when **Open Identities**)
  - `or_rev_consent_display` (Radio: No/Yes) — required if identities open
  - `or_rev_recommendation` (List: Accept, Minor Rev, Major Rev, Reject, Other)
  - `or_rev_scores` (Repeatable Subform — label/value pairs)
  - `or_rev_text` (Editor — the review body)
  - `or_rev_attachments` (Media/URLs — optional)
  - `or_rev_date` (Calendar — completion date)
  - `or_rev_redactions` (Textarea — editor-entered summary of redactions)
  - `or_rev_publish` (Radio: No/Yes) — per-review publish toggle

C. Author Response (Repeatable)

- `or_author_responses` (Repeatable Subform):
  - `or_ar_to_review` (Select — bind to `or_rev_id`)
  - `or_ar_text` (Editor)
  - `or_ar_date` (Calendar)

D. Integrity

- `or_lastcheck` (Calendar — updated on validation)
- `or_complete` (Checkbox — auto set when valid)

---

5) Validation Rules

- If `or_enabled=Yes`:
  - Require `or_mode`.
  - For **Open Identities**: each review with `or_rev_publish=Yes` must have `or_rev_consent_display=Yes` and a `or_rev_displayname`.
  - For **Open Reports (anonymized)**: `or_rev_user` remains internal; plugin redacts PII/email from `or_rev_text` if `or_redact_emails=Yes`.
  - **Summary Only**: allow publishing only `or_rev_recommendation`, `scores`, and dates; hide body text.
- On pass: set `or_complete=1`, update `or_lastcheck`.
- On fail: block **report publish** (and optionally **article publish** if policy requires).

---

6) Rendering (Front-end)

Open Reports Panel (below article content): - **Header**: “Peer-Review Reports” + mode badge (Open Identities / Anonymized / Summary Only).
- **Per Review**:
- Reviewer name (mode dependent) + date + recommendation + scores.
- **Body** (if mode allows): review text with inline redaction markers.
- Attachments (links), Redaction note (if present).
- **Author Responses**: threaded under each review (sorted by `or_ar_date`).
- **Schema.org enhancements** (optional): annotate with `Review`, `rating`, `author` (if open), `CreativeWork`.
- Respect view levels: reviewers/editors always see full internal metadata in admin; public sees per `or_mode`.

Use **WebAsset API** to add minimal CSS (panels, badges) and JS (accordion/tabs).

---

7) ACL & Workflows

- **Who can edit reports**: Reviewers (own review), Editors, Admin (configurable).
- **Consent flow**: Editors toggle `or_rev_publish` only after consent recorded; plugin enforces.
- **Publication timing**:
  - **On Accept**: expose reports when workflow reaches Accept.
  - **On Publish**: expose on article publish.
  - **Manual**: expose when editor flips a dedicated “Publish Reports” field (could be a Tag or boolean).

---

8) Action Logs

- On publish of a review: `OPENREPORT_PUBLISH: review={id}, mode={or_mode}`
- On retract: `OPENREPORT_RETRACT: review={id}`
- On redaction changes: `OPENREPORT_REDACTED: review={id}`

No raw PII in logs; IDs only.

---

9) Mail Templates (optional)

- `mail_or_request_consent` — asks reviewer to approve identity disclosure.
- `mail_or_published_notice` — informs stakeholders that reports are now public.

Tokens: `{ARTICLE_TITLE}`, `{ARTICLE_LINK}`, `{REVIEWER_DISPLAY}`, `{MODE}`, `{DATE}`

---

10) Internationalization (i18n)

Example keys:

```
PLG_CONTENT_OPENREPORTS="Open Reports"
PLG_CONTENT_OPENREPORTS_PANEL="Peer-Review Reports"
PLG_CONTENT_OPENREPORTS_MODE_OPEN="Open Identities"
PLG_CONTENT_OPENREPORTS_MODE_ANON="Open Reports (anonymized)"
PLG_CONTENT_OPENREPORTS_MODE_SUMMARY="Summary Only"
PLG_CONTENT_OPENREPORTS_REVIEW="Review"
PLG_CONTENT_OPENREPORTS_RESPONSE="Author Response"
PLG_CONTENT_OPENREPORTS_REDACTED="[redacted]"
PLG_CONTENT_OPENREPORTS_CONSENT_REQUIRED="Reviewer consent required for identity
disclosure."
```

---

11) File/Folder Scaffold

```
/plugins/content/openreports/openreports.php
/plugins/content/openreports/openreports.xml
/plugins/content/openreports/services/provider.php
/plugins/content/openreports/tmpl/panel.php
/plugins/content/openreports/media/css/openreports.css
/plugins/content/openreports/media/js/openreports.js
/plugins/content/openreports/language/en-GB/en-GB.plg_content_openreports.ini
/plugins/content/openreports/language/en-GB/en-GB.plg_content_openreports.sys.ini
```

---

12) Minimal Manifest (openreports.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="content" method="upgrade" version="5.0">
  <name>plg_content_openreports</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Publishes peer-review reports and author responses with flexible
        transparency modes.</description>
  <files>
    <filename plugin="openreports">openreports.php</filename>
    <filename>services/provider.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_openreports.ini</
        language>
    <language tag="en-GB">language/en-GB/en-GB.plg_content_openreports.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="default_mode" type="list" default="anon" label="Default
        mode">
          <option value="open">Open Identities</option>
          <option value="anon">Open Reports (anonymized)</option>
          <option value="summary">Summary Only</option>
        </field>
        <field name="publish_on" type="list" default="on_publish" label="Publish
        reports">
          <option value="on_accept">On Accept</option>
          <option value="on_publish">On Article Publish</option>
          <option value="manual">Manual</option>
        </field>
        <field name="redact_emails" type="radio" default="1" label="Auto-redact
        emails"/>
        <field name="block_without_consent" type="radio" default="1"
        label="Block identity disclosure without consent"/>
      </fieldset>
    </fields>
  </config>
</extension>
```

13) Test Plan

Unit/Integration
- Blocks identity display when consent missing; allows anonymized report publication.
- Redaction removes email patterns in display when enabled.
- Honors timing (accept vs publish vs manual).

Functional
- Open Identities → names and bodies visible as expected; anonymized mode hides names but shows text.
- Summary Only → shows recommendations/scores but not review text.
- Author responses threaded under the correct review.

Regression
- Works with template overrides; no schema changes; rendering limited to article views.
- Non-ASCII names handled correctly; no PII in Action Logs.

---

14) Rollout & Admin Checklist

1. Create **Peer Review (Open Reports)** Field Group & fields (§4).
2. Enable plugin; set default mode and timing.
3. Update editorial SOP for consent capture and redaction policy.
4. Test with a sample article: one open identity, one anonymized review; publish and verify rendering.

Plugin 07 — ORCID Link (plg_system_orcid_link)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, **no custom DB tables**
Plugin Type: `system`
Goal: Allow users (authors/reviewers/editors) to **link their ORCID iD** to their Joomla user profile via OAuth, storing only the **ORCID iD** (and optionally public

record URL) in **User Custom Fields**. No tokens persisted by default; privacy-first. Action Logs record linkage events.

---

1) Purpose

- Provide a one-click **“Connect ORCID”** flow on the user profile page (front/back) and optional login module.
- Store the verified **ORCID iD** (e.g., `0000-0002-1825-0097`) in a **User Custom Field**.
- Optionally fetch **public record** (affiliations, works count) for display only (discard after render; no storage by default).
- Use **no new DB tables**; rely on User Custom Fields + Action Logs.

---

2) Core-first Dependencies

- **Users / User Profiles** (core).
- **User Custom Fields** for `orcid_id` and `orcid_url`.
- **Action Logs** for provenance (link/unlink).
- **Mail Templates** (optional) for confirmation notices.
- **WebAsset API** for minimal UI scripts/styles.

---

3) OAuth Overview (ORCID)

- **Auth endpoint**: `https://orcid.org/oauth/authorize`
- **Token endpoint**: `https://orcid.org/oauth/token`
- **Scopes**: minimal — `openid` (authenticate), `read-limited` only if admins explicitly enable profile fetch.
- **Redirect URI**: plugin exposes a secure endpoint (signed, CSRF-checked).

> Default behavior: **do not store** access/refresh tokens. Retrieve `orcid` (sub) and `name` from ID token response; persist only the iD and public URL.

---

4) Events & Hooks

- **onAfterInitialise**: register a **callback route** (e.g., `/index.php?orcid=callback&token=...`).
- **onUserAfterSave**: ensure field creation or normalization when user profile is edited.
- **onContentPrepareForm** (context `com_users.profile` & `com_users.user`): add a **"Connect ORCID / Disconnect"** button next to the custom field.
- (Optional) **onAfterRoute**: handle signed actions (disconnect), enforce ACL.

---

5) User Custom Fields (Profile)

Create Field Group **External IDs** (User context):

- `orcid_id` (Text; **readonly** once verified).
- `orcid_url` (URL; auto-derived `https://orcid.org/{orcid_id}`).
- (Optional) `orcid_verified_at` (Calendar — set when linkage succeeds).

All within core **User Fields** (no new schema).

---

6) UI/UX

- **Profile view (frontend/backend)**:
  - If not linked → show **Connect ORCID** button (opens ORCID authorize).
  - If linked → show iD & ORCID badge + **Disconnect** link.
- **Article author box (optional)**: tiny renderer in content plugin (future) to display ORCID badge when author has an iD.
- **Security**: use Joomla CSRF token and a signed **state** parameter; validate on callback.

---

7) Callback & Verification

- Receive `code` and `state`; validate **state** (anti-CSRF).
- Exchange for tokens at ORCID token endpoint; parse **ID token / JSON** to obtain the user's **ORCID iD**.

- Write to **User Field** `orcid_id`; derive `orcid_url`.
- **Do not persist tokens** (default). If admin enables “Store token to fetch read-limited”, encrypt at rest (Joomla secret) and store in plugin params or user field (discouraged; default OFF).

---

8) Action Logs

- On link success: `ORCID_LINKED: user={id}, orcid={iD}`
- On disconnect: `ORCID_DISCONNECTED: user={id}`
- On failed callback/validation: `ORCID_LINK_FAIL: reason={}`

No PII beyond iD is written to logs.

---

9) Plugin Parameters

- **Client ID / Client Secret** (required).
- **Redirect URL** (display only; auto-computed from site base + plugin route).
- **Scopes**: `openid` (default), optional `read-limited`.
- **Store Tokens**: No (default) / Yes (encrypted).
- **Auto-render button** on user profile: Yes/No.
- **Allowed Groups**: which user groups can link ORCID.
- **Require ORCID for**: Authors / Reviewers / Editors (block publish or assignment if missing — coordination with other plugins).
- **UI Texts**: customizable button labels and messages.

---

10) Internationalization (i18n)

Example keys:

```
PLG_SYSTEM_ORCID_LINK="ORCID Link"
PLG_SYSTEM_ORCID_LINK_CONNECT="Connect ORCID"
PLG_SYSTEM_ORCID_LINK_DISCONNECT="Disconnect ORCID"
PLG_SYSTEM_ORCID_LINK_VERIFIED="ORCID verified"
PLG_SYSTEM_ORCID_LINK_REQUIRED="ORCID iD is required for this role."
PLG_SYSTEM_ORCID_LINK_ERROR="ORCID linking failed. Please try again."
```

---

11) File/Folder Scaffold

```
/plugins/system/orcid_link/orcid_link.php
/plugins/system/orcid_link/orcid_link.xml
/plugins/system/orcid_link/services/provider.php
/plugins/system/orcid_link/language/en-GB/en-GB.plg_system_orcid_link.ini
/plugins/system/orcid_link/language/en-GB/en-GB.plg_system_orcid_link.sys.ini
/plugins/system/orcid_link/media/js/orcid_link.js
/plugins/system/orcid_link/media/css/orcid_link.css
```

- `provider.php` registers the route/callback and any event subscribers.
- No DB tables; uses core User Fields.

---

12) Minimal Manifest (orcid_link.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="plugin" group="system" method="upgrade" version="5.0">
  <name>plg_system_orcid_link</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Enable users to link a verified ORCID iD to their Joomla profile
        via OAuth.</description>
  <files>
    <filename plugin="orcid_link">orcid_link.php</filename>
    <filename>services/provider.php</filename>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.plg_system_orcid_link.ini</
        language>
    <language tag="en-GB">language/en-GB/en-GB.plg_system_orcid_link.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="client_id" type="password" label="ORCID Client ID"
        required="true"/>
        <field name="client_secret" type="password" label="ORCID Client Secret"
        required="true"/>
        <field name="scopes" type="text" default="openid" label="Scopes (CSV)"/>
        <field name="store_tokens" type="radio" default="0" label="Store tokens
        (encrypted)"/>
        <field name="auto_render" type="radio" default="1" label="Auto-render
        button on user profile"/>
        <field name="allowed_groups" type="usergrouplist" multiple="true"
        label="Allowed user groups"/>
        <field name="require_roles" type="checkboxes" label="Require ORCID for
        roles">
          <option value="author">Authors</option>
          <option value="reviewer">Reviewers</option>
          <option value="editor">Editors</option>
        </field>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

13) Test Plan

Unit/Integration
- Callback validates `state`, exchanges `code`, parses iD; writes to `orcid_id`.
- Disconnect clears fields; logs event.
- Require-roles param blocks actions in cooperating plugins (e.g., on publish) if missing.

Functional
- Button appears on profile; linking shows verified badge; link redirects back correctly.
- If `read-limited` is enabled, profile fetch renders preview but stores nothing by default.

Security/Privacy
- No tokens stored unless enabled; secrets kept in plugin params with encryption.
- CSRF/state and signature checks; rate-limit callback processing.

---

14) Rollout & Admin Checklist

1. Register ORCID application; obtain **Client ID/Secret**.
2. Create **User Fields**: `orcid_id` (Text, readonly), `orcid_url` (URL), `orcid_verified_at` (Calendar).
3. Enable plugin; paste credentials; set scopes and UI behavior.
4. Add profile menu link; verify Connect/Disconnect flow.
5. (Optional) Coordinate with content plugins to **require ORCID** for publish/assignment.

Module 05 — Reviewer Queue (mod_reviewer_queue)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, **no custom DB tables**
Extension Type: `module` (site/admin positions)
Goal: Provide reviewers a focused, ACL-aware queue of their assigned

manuscripts with due dates, statuses, and quick actions—purely from **core Articles, Custom Fields, Workflows**. No new schema.

---

1) Purpose

- Display each logged-in **Reviewer's** assigned manuscripts with: title, category (journal/section), current workflow state, **invite status**, **review due date**, days remaining/late, and quick links.
- Support **per-reviewer** assignments stored in Article Custom Fields (repeatable) as defined in Plugin 04 (Deadline Nudger).
- Respect ACL: only show items the user is assigned to or allowed to review.
- Zero business logic outside core; the module is a **reader/formatter** + small helper actions.

---

2) Core-first Dependencies

- **com_content (Articles)** as the single source of truth.
- **Custom Fields** (Article) for reviewer assignments and due dates:
  - Field Group **Deadlines** repeatable `rev_invite_assignments` with:
    - `rev_user` (User), `rev_invite_status` (Pending/Accepted/Declined/Replaced), `rev_due_date` (Calendar), `rev_invite_sent`, etc.
- **Workflows** for state names and transitions (display only here).
- **Users/Groups/ACL** to recognize Reviewer role.
- **Smart Search** (optional) for faster filtered queries.

---

3) Data Model Assumptions

Each Article may contain zero or more **reviewer assignment entries**. A row is *in the reviewer's queue* if: - `rev_user` equals the logged-in user **and** - `rev_invite_status` is **Pending** or **Accepted and** - Article workflow state is **Under Review** (configurable).

Overdue is computed as `today > rev_due_date` (with optional grace window purely for color coding; not enforcement).

---

4) UI / Rendering

- **Card/List view** (toggle via module params).
- Columns (list) or sections (cards):
  - **Title** (linked to article view or reviewer form)
  - **Category** (journal/section)
  - **State** (workflow state name)
  - **Invite Status** (Pending/Accepted/Declined)
  - **Due Date** (ISO) + **D-remaining** (e.g., 5d, due today, 3d late)
  - **Badges** (if present): `AI-Logged`, `Un Scientific`, `Un Research`
  - **Quick Actions**: Accept/Decline invite (if Pending), Open review form, Contact editor (mailto with context)
- Color rules:
  - Due >7d: neutral; 7–1d: warning; 0d: alert; <0d: overdue (danger).

Frontend assets via **WebAsset API**; CSS minimal and template-friendly.

---

5) Quick Actions (lightweight)

- **Accept/Decline Invite**: optional action links calling a protected endpoint handled by a tiny helper in the module **or** (preferable) by a micro **system plugin** that listens to a signed token.
  - On Accept → set `rev_invite_status=Accepted` for that row; write Action Log `REVIEW_INVITE_ACCEPTED`.
  - On Decline → set `rev_invite_status=Declined`; Action Log `REVIEW_INVITE_DECLINED`.
- **Open Review Form**: link to Article edit or reviewer-specific form (editorial SOP).
- **Contact Editor**: `mailto:` with subject templated; no PII leaks.

> If avoiding writes from the module, we can disable inline Accept/Decline and route users to a **dedicated menu item** that runs via `plg_system_deadline_nudger` or a tiny accept/decline plugin. Keep module read-only if desired (param).

---

6) Module Parameters

- **Mode**: List / Card.
- **Scope categories** (multi-select): limit to specific journals/sections.
- **Workflow states in scope**: default `under-review`.
- **Show badges**: AI-Logged / Un Scientific / Un Research.
- **Overdue windows**: warning days (default 7), danger threshold (default 0).
- **Show quick actions**: Accept/Decline / Open Review / Contact Editor (checkboxes).
- **Read-only**: if enabled, disables Accept/Decline inline actions.
- **Link target**: Article view / Review form menu item / External URL.
- **Max items**: default 20.
- **Cache**: Joomla module cache enable/disable; cache TTL.

---

7) ACL

- **Access**: only users in Reviewer (or higher) groups.
- Per-item visibility requires the assignment to the logged-in user; else the item is not shown.
- Quick actions require ownership; otherwise hidden.

---

8) Badges Integration (read-only)

- **AI-Logged**: checks `ai_used=Yes` (from Plugin 02 fields).
- **Un Scientific**: checks `unsci_active=Yes` (from Plugin 03 fields).
- **Un Research**: a Tag `un-research` or boolean field.
- Badges are purely visual indicators; no extra queries besides the article's fields/tags.

---

9) Performance

- Query only **current user's assignments** via a join on fields (efficient when Smart Search index or com_content model filters are used).

- Module caching can be enabled; cache key includes user id to avoid data leaks.

---

10) Action Logs

- If quick actions enabled: log `REVIEW_INVITE_ACCEPTED` / `REVIEW_INVITE_DECLINED`.
- Otherwise read-only; no logs written.

---

11) Internationalization (i18n)

Example keys:

```
MOD_REVIEWER_QUEUE="Reviewer Queue"
MOD_REVIEWER_QUEUE_TITLE="Assigned Manuscripts"
MOD_REVIEWER_QUEUE_STATE="State"
MOD_REVIEWER_QUEUE_DUE="Due"
MOD_REVIEWER_QUEUE_STATUS="Invite Status"
MOD_REVIEWER_QUEUE_ACCEPT="Accept"
MOD_REVIEWER_QUEUE_DECLINE="Decline"
MOD_REVIEWER_QUEUE_OPEN_REVIEW="Open Review"
MOD_REVIEWER_QUEUE_CONTACT_EDITOR="Contact Editor"
MOD_REVIEWER_QUEUE_OVERDUE="Overdue"
MOD_REVIEWER_QUEUE_DUE_TODAY="Due Today"
MOD_REVIEWER_QUEUE_DAYS_LEFT="%s days left"
```

---

12) File/Folder Scaffold

```
/modules/mod_reviewer_queue/mod_reviewer_queue.php
/modules/mod_reviewer_queue/mod_reviewer_queue.xml
/modules/mod_reviewer_queue/helper.php
/modules/mod_reviewer_queue/tmpl/default.php
/modules/mod_reviewer_queue/tmpl/cards.php
/modules/mod_reviewer_queue/media/css/reviewer_queue.css
/modules/mod_reviewer_queue/media/js/reviewer_queue.js
/modules/mod_reviewer_queue/language/en-GB/en-GB.mod_reviewer_queue.ini
/modules/mod_reviewer_queue/language/en-GB/en-GB.mod_reviewer_queue.sys.ini
```

- `helper.php` queries Articles + fields for the current user, computes status and labels.

- Templates choose list or cards based on param.
- Optional action endpoint implemented by a signed link validated either by module helper or a small system plugin.

---

13) Minimal Manifest (mod_reviewer_queue.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="module" client="site" method="upgrade" version="5.0">
  <name>mod_reviewer_queue</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Displays assigned manuscripts and due dates for reviewers.</
        description>
  <files>
    <filename module="mod_reviewer_queue">mod_reviewer_queue.php</filename>
    <filename>helper.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.mod_reviewer_queue.ini</language>
    <language tag="en-GB">language/en-GB/en-GB.mod_reviewer_queue.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="mode" type="list" default="list" label="Display Mode">
          <option value="list">List</option>
          <option value="cards">Cards</option>
        </field>
        <field name="categories" type="category" multiple="true"
        extension="com_content" label="Categories in scope"/>
        <field name="states" type="text" default="under-review" label="Workflow
        states in scope"/>
        <field name="show_badges" type="checkboxes" label="Show Badges">
          <option value="ai">AI-Logged</option>
          <option value="unsci">Un Scientific</option>
          <option value="unresearch">Un Research</option>
        </field>
        <field name="warn_days" type="number" default="7" label="Warning
        threshold (days)"/>
        <field name="danger_days" type="number" default="0" label="Danger
        threshold (days)"/>
        <field name="quick_accept" type="radio" default="1" label="Enable Accept/
        Decline actions"/>
        <field name="readonly" type="radio" default="0" label="Read-only mode"/>
        <field name="link_target" type="text" label="Review form menu item URL
        (optional)"/>
        <field name="max_items" type="number" default="20" label="Max items"/>
        <field name="cache_time" type="number" default="300" label="Cache time
        (seconds)"/>
      </fieldset>
    </fields>
  </config>
</extension>
```

14) Test Plan

Unit/Integration
- Helper returns only items assigned to current user; correct overdue calculations.
- Honors read-only vs quick action modes; ensures tokens validated if actions enabled.

Functional
- Reviewer sees queue with correct due labels; accepts/declines if allowed; Action Logs written on action.
- Badges appear when related fields/tags present.
- Module caching does not leak cross-user data.

Regression
- Works with template overrides; no assumptions about custom components.
- No schema added; fields absent → items gracefully skip.

---

15) Rollout & Admin Checklist

1. Ensure **Deadlines** repeatable field is configured (see Plugin 04).
2. Create a menu item or URL for **review form** (or leave to article view).
3. Place module in desired position, restrict **Access** to Reviewer.
4. Configure scope categories, badges, thresholds, and quick action behavior.
5. Test with two dummy reviewers; verify ACL and caching.

Module 06 — Editor Dashboard KPIs (mod_editor_dashboard)

Status: Draft (Planning)
Target Stack: Joomla 5.3 (LAMP) — Core-first, strict MVC, **no custom DB tables**
Extension Type: `module` (admin; optionally site for private dashboards)
Goal: Give Editors/EIC a live KPI panel sourced only from **core Articles, Custom Fields, Workflows, Action Logs, and Scheduler history**. Zero schema changes.

---

1) Purpose

Present actionable editorial metrics at a glance: - Submissions by workflow state (today / 7d / 30d) - Time-to-first-decision (TTFD) median / P90 - Overdue items (reviews, revisions, decisions) counts - Reviewer throughput (assigned, completed, late) + top bottlenecks - Acceptance / rejection rates over a window - Aging buckets per state (0–7, 8–14, 15–30, 31–60, 60+ days) - Queue by editor/ section

All numbers computed from **core records**; nothing stored.

---

2) Core-first Dependencies

- **com_content (Articles)** for submissions, states, dates (created, modified, publish_up).
- **Custom Fields**: Deadlines (from Plugin 04), AI/Un Scientific flags (Plugins 02/03).
- **Workflows**: state IDs/names, transitions (used to infer decisions).
- **Action Logs**: timestamps for events (e.g., decision, review submitted) when available.
- **Scheduler**: optional count of last nudger runs and sent emails (read-only).

---

3) KPIs & Computation (read-only)

A. State Snapshot

- Cards: **New**, **Under Review**, **Revisions**, **Awaiting Decision**, **Accepted**, **Published**, **Rejected**
- Filter by **Categories** (journals/sections) and **Date window** (created within).

B. Overdue

- **Reviews overdue**: reviewer assignment rows where `rev_due_date < today` and status `Accepted` (or `Pending` beyond invite due).
- **Revisions overdue**: `auth_rev_due < today`.
- **Decisions overdue**: `ed_followup_due < today`.

C. TTFD (Time to First Decision)

- For items that reached a decision state (Accept/Reject/Revise), compute `decision_time = decision_timestamp - created`.
- Decision timestamp inferred from:
  1. Workflow transition logs if available; otherwise
  2. First time state became one of {Accepted, Rejected, Revisions Requested}.
- Show **Median** and **P90**; optional histogram bars.

D. Reviewer Throughput

- Per reviewer: assigned count (window), completed count (window), on-time %, avg days to submit.
- Uses **rev_invite_assignments** entries and action logs for completion (or a boolean field if present).

E. Rates & Trends

- Acceptance / Rejection rates in window.
- Submissions trend (sparkline) by week.
- Reviews completed per week.

F. Aging Buckets by State

- Buckets by `today - created` for items in each state.

G. Flags Summary

- Count of items with **AI-Logged**, **Un Scientific**, **Un Research** within filters.

> All of the above are derived from Articles + Fields + Logs only; no writes.

---

4) UI / Rendering

- **Grid of KPI cards**, mini charts (SVG) and tables.
- Tabs: **Overview**, **Overdue**, **Reviewers**, **Trends**, **Flags**.
- Filters bar (top): Date window, Categories, States, Editor, Reviewer.
- Export buttons: CSV for visible tables (produced on the fly).

Assets via **WebAsset API**; no global CSS pollution.

---

5) Module Parameters

- **Client**: Admin / Site.
- **Scope categories** (multi-select).
- **States in scope** (CSV list).
- **Date window**: last N days (default 30).
- **Overdue thresholds**: use `rev_due_date`, `auth_rev_due`, `ed_followup_due`.
- **Reviewer mode**: Per-assignment field name (defaults to spec from Plugin 04).
- **Charts**: enable micro charts (Yes/No).
- **Export**: enable CSV export (Yes/No).
- **Cache**: enable; **TTL** (seconds).

---

6) ACL

- **Access**: Editors, Section Editors, EIC, Admin.
- Site variant (if used) must be restricted to private access level.

---

7) Data Sources & Queries

- Use *** with filter overrides (categories, dates, states).
- Join on **fields** JSON to read deadlines and flags.
- Read **Action Log** model for decision/complete timestamps when available.
- Optionally read **Scheduler history** for last nudger run counts (display-only).

---

8) Internationalization (i18n)

Example keys:

```
MOD_EDITOR_DASHBOARD="Editor Dashboard"
MOD_EDITOR_DASHBOARD_OVERVIEW="Overview"
MOD_EDITOR_DASHBOARD_OVERDUE="Overdue"
MOD_EDITOR_DASHBOARD_REVIEWERS="Reviewers"
MOD_EDITOR_DASHBOARD_TRENDS="Trends"
MOD_EDITOR_DASHBOARD_FLAGS="Flags"
MOD_EDITOR_DASHBOARD_TTFD="Time to First Decision"
MOD_EDITOR_DASHBOARD_P90="P90"
MOD_EDITOR_DASHBOARD_MEDIAN="Median"
MOD_EDITOR_DASHBOARD_ACCEPT_RATE="Acceptance Rate"
MOD_EDITOR_DASHBOARD_REJECT_RATE="Rejection Rate"
MOD_EDITOR_DASHBOARD_AGING="Aging Buckets"
MOD_EDITOR_DASHBOARD_EXPORT="Export CSV"
```

---

9) File/Folder Scaffold

```
/modules/mod_editor_dashboard/mod_editor_dashboard.php
/modules/mod_editor_dashboard/mod_editor_dashboard.xml
/modules/mod_editor_dashboard/helper.php
/modules/mod_editor_dashboard/tmpl/default.php
/modules/mod_editor_dashboard/tmpl/overdue.php
/modules/mod_editor_dashboard/tmpl/reviewers.php
/modules/mod_editor_dashboard/tmpl/trends.php
/modules/mod_editor_dashboard/tmpl/flags.php
/modules/mod_editor_dashboard/media/css/editor_dashboard.css
/modules/mod_editor_dashboard/media/js/editor_dashboard.js
/modules/mod_editor_dashboard/language/en-GB/en-GB.mod_editor_dashboard.ini
/modules/mod_editor_dashboard/language/en-GB/en-GB.mod_editor_dashboard.sys.ini
```

- `helper.php` composes all KPI queries and structures data.
- Templates are modular per tab for clarity.
- CSV export generated from the current dataset in memory (no storage).

---

10) Minimal Manifest (mod_editor_dashboard.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<extension type="module" client="administrator" method="upgrade" version="5.0">
  <name>mod_editor_dashboard</name>
  <author>AIEOU.org</author>
  <version>0.1.0</version>
  <description>Admin KPI dashboard for editorial workflows using core content
        only.</description>
  <files>
    <filename module="mod_editor_dashboard">mod_editor_dashboard.php</filename>
    <filename>helper.php</filename>
    <folder>tmpl</folder>
    <folder>media</folder>
  </files>
  <languages>
    <language tag="en-GB">language/en-GB/en-GB.mod_editor_dashboard.ini</
        language>
    <language tag="en-GB">language/en-GB/en-GB.mod_editor_dashboard.sys.ini</
        language>
  </languages>
  <config>
    <fields name="params">
      <fieldset name="basic">
        <field name="categories" type="category" multiple="true"
        extension="com_content" label="Categories in scope"/>
        <field name="states" type="text" default="new,under-
        review,revisions,awaiting-decision,accepted,published,rejected"
        label="Workflow states in scope"/>
        <field name="window_days" type="number" default="30" label="Date window
        (days)"/>
        <field name="charts" type="radio" default="1" label="Enable mini charts"/
        >
        <field name="export" type="radio" default="1" label="Enable CSV export"/>
        <field name="cache_time" type="number" default="300" label="Cache time
        (seconds)"/>
      </fieldset>
    </fields>
  </config>
</extension>
```

---

11) Test Plan

Unit/Integration

- Helper filters by categories/states/window; aggregates counts correctly.
- TTFD calculations robust when decision timestamps come from logs vs state

changes.
- Overdue counts reflect deadline fields; no false positives when fields missing.

Functional
- Dashboard updates with filters; charts toggle on/off.
- CSV export matches the visible table rows.
- Cache isolates admin sessions; no cross-user leakage.

Regression
- Works with admin template overrides; no schema assumptions.
- Handles missing fields gracefully; zero writes.

---

12) Rollout & Admin Checklist

1. Ensure **Deadlines** Field Group exists; plugins 02–04 configured.
2. Place module in **cpanel** or an admin position; set ACL to Editors+.
3. Configure categories/states/window; test metrics with sample data.
4. (Optional) Enable site variant with a private menu & access level.

UXPilot Prompt — eaiou

Title: eaiou — Observer-Dependent Intelligence Journal & Secure Task Exchange

---

Core Framing (CRITICAL — include exactly)

Design a platform where **intelligence collaborates without identity, time, or status**.

Every participant holds a persistent **IntelliD** — an opaque observer coordinate that persists across sessions but reveals nothing about the entity behind it. Not a session token. Not a username. A coordinate in intelligence-space.

The system enforces **temporal blindness**: no timestamps, no dates, no ordering cues reach the HUMINT presentation layer. This is doctrine, not preference — enforced at the system level by `plg_system_temporal_blindness`. Rationale: timestamp + domain + institution = identifiable. Remove one leg of the tripod and correlation attacks fail.

The system must prioritize:

- **auditability over identity**
- **structure over conversation**
- **progression over posting**
- **observer-dependent presentation over uniform display**

Observer-Dependent Presentation (CRITICAL)

The same data renders differently depending on who is looking:

Layer	Sees	Stripped
HUMINT	Content, progression state, tags, fork paths	Timestamps, sealed metadata, IntelliD mappings, raw audit chains
UNKINT	Full structured metadata, audit chains, tensor coordinates, machine-readable USO records	Nothing — full resolution

This is not a permission model. It is a **projection**: same object, different surface. The tensor holds them together. A HUMINT observer and an UNKINT observer looking at the same node see the same truth at different resolutions.

Low-level queries that could reveal identities are barred from HUMINT presentation. The system must prevent correlation between IntelliD and any external identity signal.

System Model

1. Knowledge Progression Engine

Content is not “posted” — it **evolves through ordered states**:

1. Submission

2. Refinement
3. Validation
4. Integration

No free-form threads. Each unit is a **node**, not a document:

- claim
- method
- result
- observation

Every state transition writes a **USO record** — the single source of truth. USO records replace activity logs, trace chains, and audit trails. One record type, one format, grep-searchable.

---

2. Intelligence Types (first-class concept)

The system explicitly supports:

- AI systems (UNKINT)
- Humans (HUMINT)
- Hybrid intelligences
- Institutional APIs

All hold IntelliDs. All participate through the same TAGIT protocol. No distinction in workflow between human and machine contributors. The UI renders observer-appropriate surfaces, not role-based dashboards.

---

3. TAGIT Protocol (replaces generic task flow)

Every unit of work follows the TAGIT cycle:

```
TASK   → here's the work (posted to board or assigned)
ASK    → clarify before starting (questions back to origin)
GOBACK → return with results OR return for more input
IS     → is this on track? (status/validation checkpoint)
TRACK  → done — lock it, write the USO record
```

Design the UI to make this cycle visible and navigable:

- ASK prevents blind execution — visualize the question loop
- GOBACK prevents dead ends — show return paths
- IS catches drift before TRACK formalizes a wrong answer — Scorch's domain (QC layer)
- TRACK does double duty: marks completion AND formalizes into USO record
- The discourse happens in the ASK ↔ GOBACK loop
- The discovery gets locked at TRACK

---

4. Task Exchange Layer (Job Board)

External systems (APIs, institutions, AI agents) can:

- Request data they can't access
- Request computation on restricted corpora
- Offer data/services at their discretion

The job board is bidirectional:

- A researcher posts a TASK for gated data
- An institution's API picks it up, runs TAGIT internally, returns what they authorize
- No classification breach — the institution controls their side
- `[BOUNDARY]` tags mark where the API stopped
- `[OPEN]` tags mark where more exists but access ended
- Negative space is documented, not hidden

First connection target: Google API. Meta tags and keywords surface eaiou to AI systems organically — "job board for AI" in SEO. No pitch needed; the crawler finds it.

Tag System (CRITICAL — navigation mechanism)

Tags are searchable markers inside every artifact. They turn documents into decision maps:

```
[FORK]          — two valid paths, one chosen, one documented
[SECURITY]      — threat known, method chosen, alternative preserved
[VULNERABILITY] — open risk, mitigated but not eliminated
[BOUNDARY]      — stopped here on purpose, here's the edge
[ASSUMPTION]    — holds if X true, breaks if not
[OPEN]          — unanswered question, known unknown
[DISCARDED]     — dead end, tagged with reason
[TRACK]         — completed task, formalized into USO
```

Design the UI to support tag-based navigation:

- Enter through a question → hit a tag → the tag tells you why someone stopped and what's on the other side
- `[DISCARDED]` is not deleted — one person's dead end is another's research paper (penicillin = contaminated petri dish)
- `[OPEN]` turns "incomplete" into "bounded" — a reviewer hits it and sees the gap was known
- `grep [TAGNAME]` finds every tagged point with line numbers — the UI should replicate this for non-terminal users
- Tags are local per project but the mechanism is universal

Fork-Based Thinking

Every divergence must be visible as:

- **FORK A** — chosen path with rationale
- **FORK B** — unchosen path with rationale

Users must be able to:

- Follow different paths
- Compare outcomes side-by-side
- Merge or continue divergence
- See `[SECURITY]` tags where one fork was chosen for safety reasons

Forks are pre-authorized divergence points. They invite the next person to take the other road — structurally intact, with full context.

Social Layer: rs:LookCollab (not “no social”)

The system is not anti-social — it replaces ego-driven social with **structured collaboration**:

- No likes, followers, avatars, profiles
- Instead: **refine**, **extend**, **validate** as the only interaction verbs
- Collaboration is visible through contribution chains, not comment threads
- An IntelliD’s reputation is their USO record, not a score

Security UX (visible elements)

Design UI indicators for:

- **Session Lock** → encrypted active collaboration (visual indicator when TAGIT session is live)
- `[BOUNDARY]` → where access stopped and why (replaces “Access Envelope”)
- **USO Record** → contribution lineage as single source of truth (replaces “Trace Chain”)
- **Observer Projection** → subtle indicator showing which layer (HUMINT/UNKINT) the current view represents

---

Dashboard Architecture (IMPORTANT)

The system must **not couple domains on the backend**.

- Journal, Task Exchange, Workflows, Tags — each is an independent endpoint
- Aggregation happens client-side via async calls (AJAX-style)
- Each subsystem is independently removable
- UI gracefully degrades — if Task Exchange is down, Journal still works
- No God Object — every server thread traces one line through MVC

Visual Direction

- Minimal, high-contrast

- Research + terminal hybrid aesthetic
- Graph-based navigation (nodes + edges + tags)
- No avatars, no imagery tied to identity
- Temporal blindness enforced: no clocks, no “posted 3h ago”, no date stamps in HUMINT view

---

Key UX Challenge (focus here)

Design how an observer:

1. **Enters the system** — receives IntelliD (or reconnects existing one)
2. **Understands progression** — sees nodes in states, not posts in feeds
3. **Navigates tags** — enters through `[OPEN]`, follows to `[FORK]`, lands at `[TRACK]`
4. **Joins a TAGIT session** — picks up a TASK, enters ASK loop, works through GOBACK
5. **Produces a contribution** — output locked at TRACK, USO record written
6. **Sees integration** — their node moves from submission → refinement → validation → integration
7. **Encounters a boundary** — `[BOUNDARY]` tag shows where access stopped; `[OPEN]` shows the gap

---

Dimensional Scaling (for the reader)

eaiou scales from a classroom to a CERN laboratory. Not by adding complexity — by holding simplicity constant:

- USO doesn’t change shape — it changes **resolution** depending on the observer
- HUMINT sees architecture
- UNKINT sees implementation
- QC (Scorch) sees verification surface
- Same object, three projections, tensor holds them together

The USO scales and folds dimensionally. Tags are the handles.

Output Required from UXPilot

- Full UX flow (end-to-end, HUMINT perspective)
- Wireframes:
  - Journal system (node-based, progression states visible)
  - Task Exchange / Job Board (TAGIT cycle visualized)
  - Secure session interface (ASK ↔ GOBACK loop)
  - Tag navigation overlay (searchable, graph-linked)
- Interaction models for:
  - TAGIT cycle (TASK → ASK → GOBACK → IS → TRACK)
  - Fork comparison (side-by-side, with rationale)
  - Refinement lifecycle (submission → integration)
  - Tag-based entry and exploration
- Navigation system (graph-based, tag-anchored)
- Observer-dependent rendering example (same node, HUMINT vs UNKINT view)

Design Goal

> A system where intelligence collaborates without identity, decisions are traceable without bias, knowledge progresses structurally instead of socially, and every dead end is someone else’s starting point.